

第9章 推理优化

新模型不断涌现，但有一点始终不变：如何使它们更好、更便宜、更快。到目前为止，本书已经讨论了各种让模型更好的技术。本章将重点关注如何让模型更快、更便宜。

无论你的模型有多好，如果它太慢，用户可能会失去耐心，或者更糟糕的是，其预测可能变得毫无用处——想象一下一个需要两天时间来计算每个结果的次日股价预测模型。如果你的模型太昂贵，其投资回报将不值得。

推理优化可以在模型、硬件和服务层面进行。在模型层面，你可以减小训练模型的大小或开发更高效的架构，比如没有transformer模型中常见的注意力机制计算瓶颈的架构。在硬件层面，你可以设计更强大的硬件。

推理服务在给定硬件上运行模型以满足用户请求。它可以采用为特定硬件优化模型的技术。它还需要考虑使用和流量模式，以有效分配资源来减少延迟和成本。

因此，推理优化是一个跨学科领域，通常需要模型研究人员、应用开发人员、系统工程师、编译器设计师、硬件架构师甚至数据中心运营商之间的合作。

本章讨论AI推理的瓶颈以及克服这些瓶颈的技术。它将主要关注模型和服务层面的优化，并概述AI加速器。

本章还涵盖性能指标和权衡。有时，加速模型的技术也可以降低其成本。例如，降低模型的精度使其更小、更快。但通常，优化需要权衡。例如，最佳硬件可能使模型运行更快，但成本更高。

随着开源模型的日益普及，更多团队正在构建自己的推理服务。然而，即使你不实施这些推理优化技术，理解这些技术也将帮助你评估推理服务和框架。如果你的应用程序的延迟和成本对你造成影响，请继续阅读。本章可能会帮助你诊断原因和潜在解决方案。

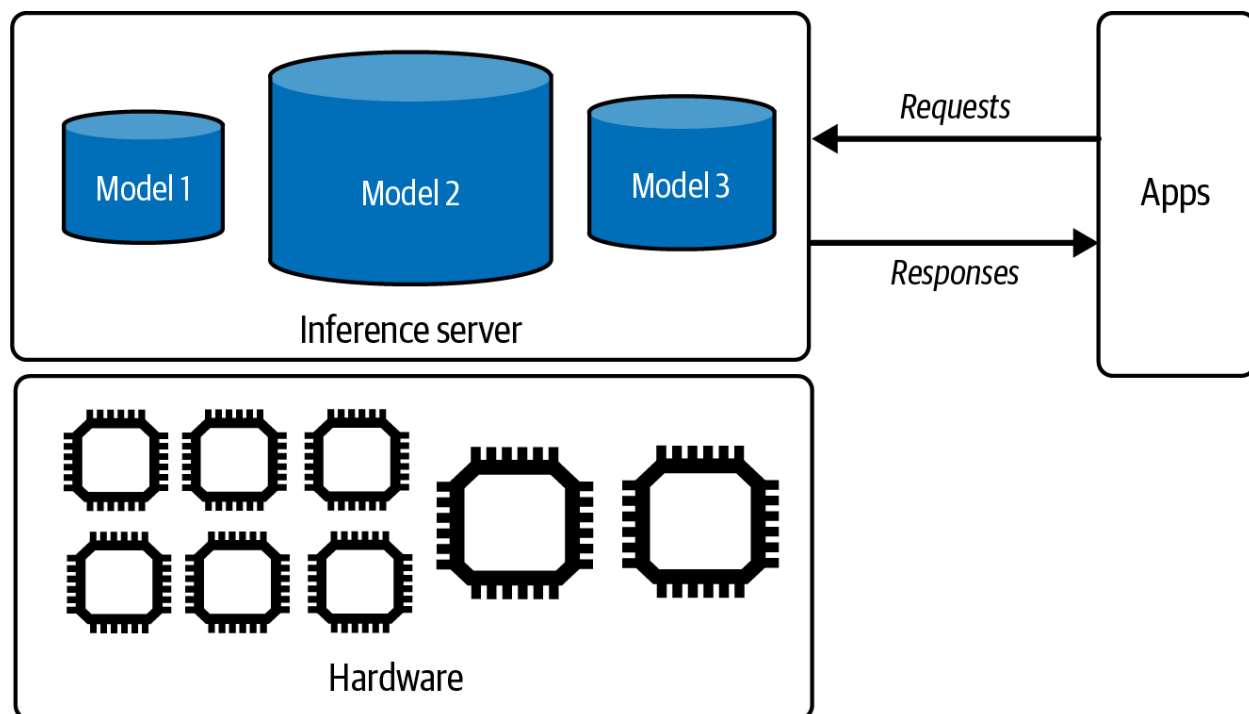
理解推理优化

AI模型生命周期中有两个不同的阶段：训练和推理。训练是指构建模型的过程。推理是指使用模型为给定输入计算输出的过程。除非你训练或微调模型，否则你主要需要关心推理。

本节首先概述推理，介绍共享词汇以讨论本章的其余部分。如果你已经熟悉这些概念，可以随时跳到感兴趣的部分。

推理概述

在生产环境中，运行模型推理的组件称为推理服务器。它托管可用的模型并可访问必要的硬件。基于来自应用程序的请求（例如，用户提示），它分配资源执行适当的模型并将响应返回给用户。推理服务器是更广泛的推理服务的一部分，该服务还负责接收、路由和可能在请求到达推理服务器之前对其进行预处理。图9-1显示了一个简单的推理服务。



像OpenAI和Google提供的模型API都是推理服务。如果你使用这些服务之一，你将不需要实现本章讨论的大多数技术。然而，如果你自己托管模型，你将负责构建、优化和维护其推理服务。

计算瓶颈

优化是关于识别瓶颈并解决它们。例如，为了优化交通，城市规划者可能会识别拥堵点并采取措施缓解拥堵。同样，推理服务器应设计为解决其服务的推理工作负载的计算瓶颈。主要有两种计算瓶颈，计算受限和内存带宽受限：

计算受限 这指的是完成时间由任务所需的计算决定的任务。例如，密码解密通常是计算受限的，由于破解加密算法需要密集的数字计算。

内存带宽受限 这些任务受到系统内数据传输率的限制，例如内存和处理器之间的数据移动速度。例如，如果你将数据存储在CPU内存中并在GPU上训练模型，则必须将数据从CPU移动到GPU，这可能需要很长时间。这可以缩写为带宽受限。在文献中，内存带宽受限通常被称为内存受限。

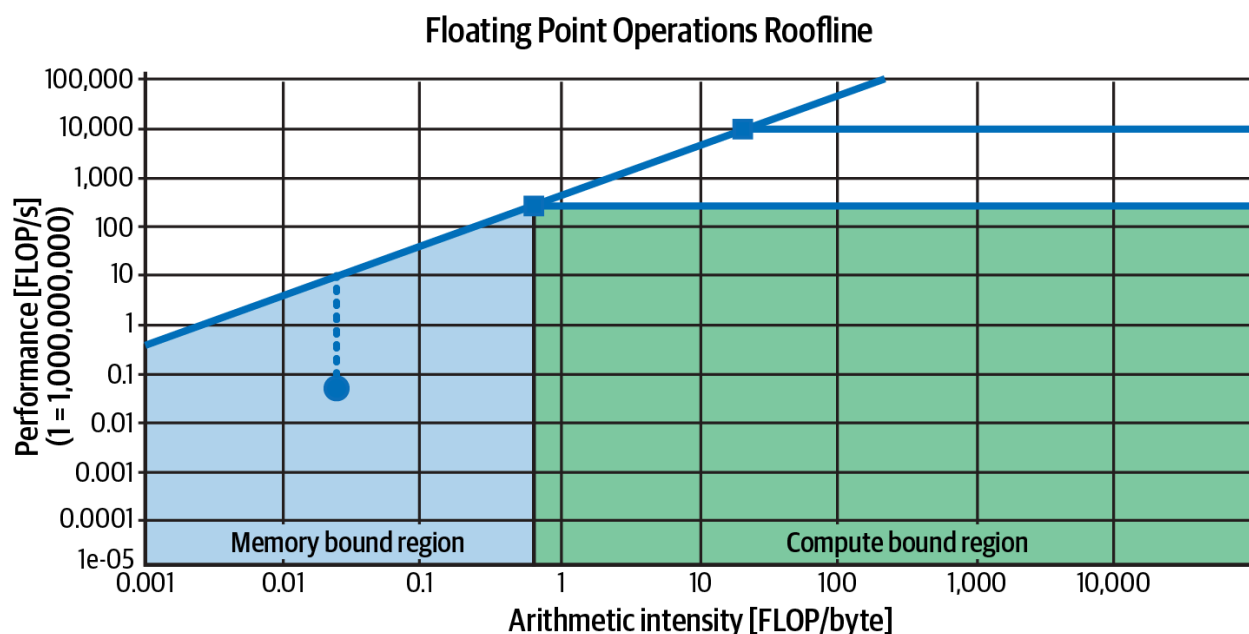
术语歧义：内存受限与带宽受限 内存受限也被一些人用来指完成时间受内存容量而非内存带宽限制的任务。当你的硬件没有足够的内存来处理任务时，就会发生这种情况，例如，如果你的机器

没有足够的内存来存储整个互联网。这种内存限制通常表现为工程师们都熟悉的错误:OOM(内存不足)。

然而,这种情况通常可以通过将任务分成更小的部分来缓解。例如,如果你受到GPU内存的限制并且无法将整个模型放入GPU,你可以将模型分散在GPU内存和CPU内存之间。这种分割会减慢你的计算,因为将数据在CPU和GPU之间传输需要时间。然而,如果数据传输足够快,这就不那么成问题了。因此,内存容量限制实际上更多地是关于内存带宽。

计算受限或内存带宽受限的概念是在《屋顶线》论文(Williams等, 2009)中引入的。从数学上讲,一个操作可以根据其算术强度(每字节内存访问的算术操作数)被分类为计算受限或内存带宽受限。像NVIDIA Nsight这样的分析工具会显示一个屋顶线图表来告诉你的工作负载是计算受限还是内存带宽受限,如图9-2所示。这个图表称为屋顶线图表,因为它看起来像一个屋顶。屋顶线图表在硬件性能分析中很常见。

不同的优化技术旨在缓解不同的瓶颈。例如,计算受限的工作负载可能通过将其分散到更多芯片或利用计算能力更强的芯片(例如,具有更高FLOP/s数字的芯片)来加速。内存带宽受限的工作负载可能通过利用具有更高带宽的芯片来加速。



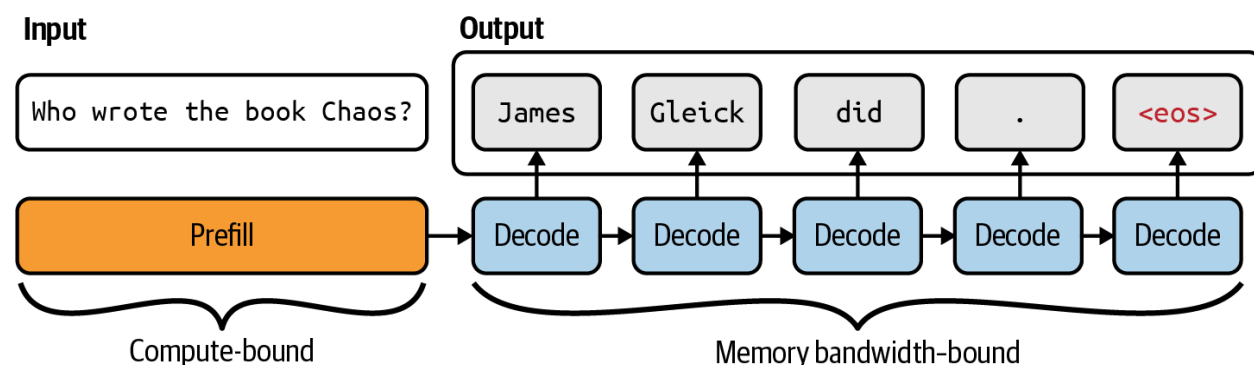
不同的模型架构和工作负载导致不同的计算瓶颈。例如,像Stable Diffusion这样的图像生成器的推理通常是计算受限的,而自回归语言模型的推理通常是内存带宽受限的。

作为说明,让我们看看语言模型推理。回顾第2章,基于transformer的语言模型推理包括两个步骤,预填充和解码:

预填充 模型并行处理输入标记。一次可以处理多少个标记受到你的硬件在给定时间内可以执行的操作数量的限制。因此,预填充是计算受限的。

解码 模型一次生成一个输出标记。在高层次上，这一步通常涉及将大型矩阵(例如，模型权重)加载到GPU中，这受到你的硬件可以多快将数据加载到内存中的限制。因此，解码是内存带宽受限的。

图9-3可视化了预填充和解码。



由于预填充和解码具有不同的计算特性，它们在生产中通常被解耦，使用单独的机器。这种技术将在"推理服务优化"中讨论。

影响LLM推理服务器中预填充和解码计算量(因此其瓶颈)的因素包括上下文长度、输出长度和请求批处理策略。长上下文通常导致内存带宽受限的工作负载，但本章后面讨论的巧妙优化技术可以消除这一瓶颈。

截至撰写本文时，由于transformer架构的普及和现有加速器技术的限制，许多AI和数据工作负载都是内存带宽受限的。然而，未来的软件和硬件进步将能够使AI和数据工作负载成为计算受限的。

在线和批量推理API

许多提供商提供两种类型的推理API，在线和批量：

在线API优化延迟。请求一到达就立即处理。

批量API优化成本。如果你的应用程序没有严格的延迟要求，你可以将它们发送到批量API以获得更高效的处理。更高的延迟允许更广泛的优化技术，包括将请求批量处理在一起和使用更便宜的硬件。例如，截至撰写本文时，Google Gemini和OpenAI都提供了批量API，成本降低了50%，周转时间显著更高，即以小时而非秒或分钟计。

即使在线API也可能将请求批量处理在一起，只要不显著影响延迟，如"批处理"中所讨论的。唯一的区别是，在线API侧重于降低延迟，而批量API侧重于提高吞吐量。

面向客户的用例，如聊天机器人和代码生成，通常需要较低的延迟，因此倾向于使用在线API。延迟要求不那么严格的应用例，适合批量API的包括以下内容：

- 合成数据生成
- 周期性报告，如总结Slack消息、社交媒体上品牌提及的情感分析以及分析客户支持工单
- 新客户入职，需要处理所有上传的文档
- 迁移到需要重新处理所有数据的新模型
- 为大型客户群生成个性化推荐或通讯
- 通过重新索引组织的数据更新知识库

API通常默认返回完整响应。然而，使用自回归解码，模型完成响应可能需要很长时间，而用户不耐烦。许多在线API提供流模式，在生成每个标记时返回它。这减少了用户等待第一个标记的时间。这种方法的缺点是你不能在向用户显示响应之前对其进行评分，增加了用户看到不良响应的风险。然而，你仍然可以在检测到风险后立即回顾性地更新或删除响应。

警告 基础模型的批量API与传统ML的批量推理不同。在传统ML中：

- 在线推理意味着在请求到达后计算预测。
- 批量推理意味着在请求到达之前预先计算预测。

预计算对于具有有限且可预测输入的用例(如推荐系统)是可能的，其中可以提前为所有用户生成推荐。这些预先计算的预测会在请求到达时被获取，例如，当用户访问网站时。然而，对于输入是开放式的基础模型用例，很难预测所有用户提示。

推理性能指标

在进入优化之前，理解要优化的指标很重要。从用户角度看，中心轴是延迟(响应质量是模型本身的特性，而非推理服务的特性)。然而，应用程序开发人员还必须考虑吞吐量和利用率，因为它们决定了应用程序的成本。

延迟、TTFT和TPOT

延迟衡量从用户发送查询到接收完整响应的时间。对于自回归生成，特别是在流模式下，总体延迟可以分解为几个指标：

首个标记的时间 TTFT衡量用户发送查询后第一个标记生成的速度。它对应于预填充步骤的持续时间，取决于输入的长度。用户对不同应用程序的TTFT可能有不同的期望。例如，对于会话型聊天机器人，TTFT应该是瞬时的。然而，用户可能愿意等待更长时间来总结长文档。

每输出标记的时间 TPOT衡量第一个标记之后每个输出标记生成的速度。如果每个标记需要100毫秒，1,000个标记的响应将需要100秒。

在流模式下，用户在生成每个标记时阅读它，TPOT应该比人类阅读速度快，但不需要快得多。一个非常快的读者可以读120毫秒/标记，所以对于大多数用例，大约120毫秒的TPOT，或每秒6-8个标记，就足够了。

标记之间的时间和标记间延迟 这个指标的变体包括标记之间的时间(TBT)和标记间延迟(ITL)。两者都衡量输出标记之间的时间。

总延迟将等于 $TTFT + TPOT \times (\text{输出标记数量})$ 。

具有相同总延迟的两个应用程序可以通过不同的TTFT和TPOT提供不同的用户体验。你的用户是喜欢即时的第一个标记, 然后标记之间有更长的等待, 还是喜欢稍微等待更长时间的第一个标记, 但之后享受更快的标记生成? 用户研究将有助于确定最佳用户体验。通过将更多计算实例从解码转移到预填充, 可以以更高的TPOT为代价来降低TTFT, 反之亦然。

重要的是要注意, 用户观察到的TTFT和TPOT值可能与模型观察到的不同, 特别是在涉及CoT(思维链)或代理查询的场景中, 模型生成中间步骤但不向用户显示。一些团队使用"发布时间"指标, 明确表示它衡量的是用户看到的第一个标记的时间。

考虑以下场景, 用户发送查询后, 模型执行以下步骤:

1. 生成一个计划, 包括一系列操作。这个计划不向用户显示。
2. 执行操作并记录其输出。这些输出不向用户显示。
3. 基于这些输出, 生成显示给用户的最终响应。

从模型的角度看, 第一个标记在步骤1中生成。这是模型内部开始其标记生成过程的时间。然而, 用户只看到在步骤3中生成的最终输出的第一个标记。因此, 从他们的角度看, TTFT要长得多。

因为延迟是一个分布, 平均值可能会产生误导。想象你有10个请求, TTFT值分别是100毫秒、102毫秒、100毫秒、100毫秒、99毫秒、104毫秒、110毫秒、90毫秒、3,000毫秒、95毫秒。平均TTFT值是390毫秒, 这使你的推理服务看起来比实际情况慢。可能是网络错误减慢了一个请求, 或者是特别长的提示需要更长时间来预填充。无论如何, 你都应该进行调查。随着请求数量的增加, 几乎不可避免地会出现扭曲平均延迟的异常值。

查看百分位数延迟更有帮助, 因为它们告诉你关于一定百分比请求的信息。最常见的百分位数是第50百分位, 简称p50(中位数)。如果中位数是100毫秒, 那么一半的请求生成第一个标记需要超过100毫秒, 一半需要少于100毫秒。百分位数也有助于发现异常值, 这可能是出现问题的症状。通常, 你想查看的百分位数是p90、p95和p99。将TTFT值与输入长度绘制在一起也很有帮助。

吞吐量 and 有效吞吐量

吞吐量衡量推理服务在所有用户和请求中每秒可以生成的输出标记数量。

一些团队在吞吐量计算中同时计算输入和输出标记。然而, 由于处理输入标记(预填充)和生成输出标记(解码)有不同的计算瓶颈, 并且在现代推理服务器中通常被解耦, 输入和输出吞吐量应该分开计算。当使用吞吐量而没有任何修饰符时, 它通常指的是输出标记。

吞吐量通常以标记/秒(TPS)来衡量。如果你服务多个用户, 标记/秒/用户也用于评估系统如何随着更多用户扩展。

吞吐量也可以衡量为在给定时间内完成的请求数量。许多应用程序使用每秒请求数(RPS)。然而,对于构建在基础模型之上的应用程序,一个请求可能需要几秒钟才能完成,所以许多人使用每分钟完成的请求数(RPM)来代替。跟踪此指标对了解推理服务如何处理并发请求很有用。如果你同时发送太多并发请求,一些提供商可能会限制你的服务。

吞吐量与计算成本直接相关。更高的吞吐量通常意味着更低的成本。如果你的系统在计算方面每小时花费2美元,其吞吐量为100标记/秒,则每100万输出标记的成本大约为5.556美元。如果每个请求平均生成200个输出标记,解码1,000个请求的成本将是1.11美元。

预填充成本可以类似计算。如果你的硬件每小时花费2美元,它每分钟可以预填充100个请求,预填充1,000个请求的成本将是0.33美元。

每个请求的总成本是预填充和解码成本的总和。在这个例子中,1,000个请求的总成本将是1.11美元 + 0.33美元 = 1.44美元。

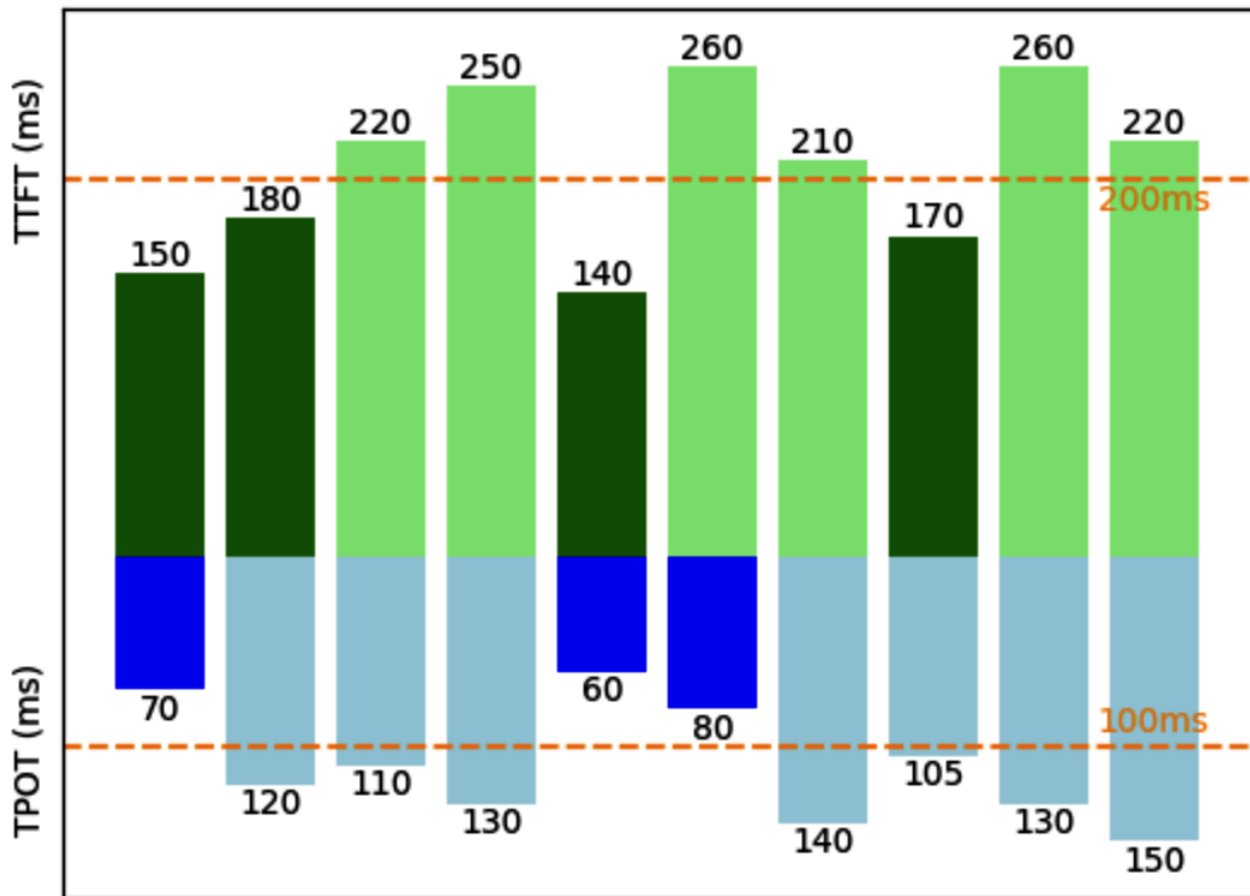
什么被认为是好的吞吐量取决于模型、硬件和工作负载。较小的模型和高端芯片通常会导致更高的吞吐量。具有一致输入和输出长度的工作负载比具有可变长度的工作负载更容易优化。

即使对于类似大小的模型、硬件和工作负载,直接吞吐量比较可能只是近似的,因为标记数量取决于什么构成一个标记,而不同的模型有不同的分词器。使用诸如每请求成本之类的指标来比较推理服务器的效率更好。

就像大多数其他软件应用程序一样, AI应用程序有延迟/吞吐量的权衡。批处理等技术可以提高吞吐量,但会减少延迟。根据LinkedIn AI团队在部署生成式AI产品一年后的反思(LinkedIn, 2024),如果你愿意牺牲TTFT和TPOT,将吞吐量增加两到三倍并不罕见。

由于这种权衡,仅基于其吞吐量和成本来关注推理服务可能导致糟糕的用户体验。相反,一些团队关注有效吞吐量,这是从网络领域改编过来的LLM应用程序指标。有效吞吐量衡量满足SLO(软件级别目标)的每秒请求数量。

想象一下,你的应用程序有以下目标:TTFT最多200毫秒,TPOT最多100毫秒。假设你的推理服务每分钟可以完成100个请求。然而,在这100个请求中,只有30个满足SLO。那么,该服务的有效吞吐量是每分钟30个请求。图9-4显示了这一点的可视化。



【图9-4. 如果推理服务每秒可以完成10个请求，但只有3个满足SLO，那么其有效吞吐量是每秒3个请求。】

利用率、MFU和MBU

利用率指标衡量资源使用的效率。它通常量化正在使用的资源与其总可用容量相比的比例。

一个常见但经常被误解的指标是GPU利用率，NVIDIA对此误解负有部分责任。NVIDIA官方用于监控GPU使用情况的工具是nvidia-smi——SMI代表系统管理接口。这个工具显示的一个指标是GPU利用率，它表示GPU积极处理任务的时间百分比。例如，如果你在GPU集群上运行推理10小时，而GPU积极处理任务5个小时，你的GPU利用率将是50%。

然而，积极处理任务并不意味着高效处理。简单起见，考虑一个能够每秒执行100个操作的微型GPU。在nvidia-smi的利用率定义中，这个GPU可以报告100%的利用率，即使它每秒只执行一个操作。

如果你为一台能够执行100个操作的机器付费，但只用它执行1个操作，你就是在浪费钱。因此，nvidia-smi的GPU优化指标不是很有用。你可能关心的利用率指标是，在机器能够计算的所有操

作中，它在给定时间内执行了多少。这个指标称为MFU(模型FLOP/s利用率)，以将其与NVIDIA GPU利用率指标区分开来。

MFU是观察到的吞吐量(标记/秒)相对于系统在峰值FLOP/s下运行的理论最大吞吐量的比率。如果在芯片制造商宣传的峰值FLOP/s下，芯片可以生成100标记/秒，但当用于你的推理服务时，它只能生成20标记/秒，你的MFU是20%。

同样，因为内存带宽是昂贵的，你可能还想知道你的硬件的带宽被利用得有多有效。MBU(模型带宽利用率)衡量所使用的可达到内存带宽的百分比。如果芯片的峰值带宽是1 TB/s，而你的推理只使用500 GB/s，你的MBU是50%。

计算LLM推理使用的内存带宽很简单：

参数数量 × 每参数字节数 × 标记/秒

MBU计算如下：

$(\text{参数数量} \times \text{每参数字节数} \times \text{标记/秒}) / (\text{理论带宽})$

例如，如果你使用FP16(每参数两个字节)的7B参数模型并达到100标记/秒，使用的带宽是：

$7\text{B} \times 2 \times 100 = 700 \text{ GB/s}$

这强调了量化(在第7章中讨论)的重要性。每参数更少的字节意味着你的模型消耗更少的宝贵带宽。

如果这是在理论内存带宽为2 TB/s的A100-80GB GPU上完成的，MBU是：

$(700 \text{ GB/s}) / (2 \text{ TB/s}) = 70\%$

吞吐量(标记/秒)与MBU之间的关系以及吞吐量与MFU之间的关系是线性的，所以有些人可能使用吞吐量来指代MBU和MFU。

什么被认为是好的MFU和MBU取决于模型、硬件和工作负载。计算受限的工作负载通常具有更高的MFU和更低的MBU，而带宽受限的工作负载通常显示更低的MFU和更高的MBU。

由于训练可以受益于更有效的优化(例如，更好的批处理)，这要归功于更可预测的工作负载，训练的MFU通常高于推理的MFU。对于推理，由于预填充是计算受限的，而解码是内存带宽受限的，预填充期间的MFU通常高于解码期间的MFU。对于模型训练，截至撰写本文时，超过50%的MFU通常被认为是好的，但在特定硬件上可能很难实现。表9-1显示了几个模型和加速器的MFU。

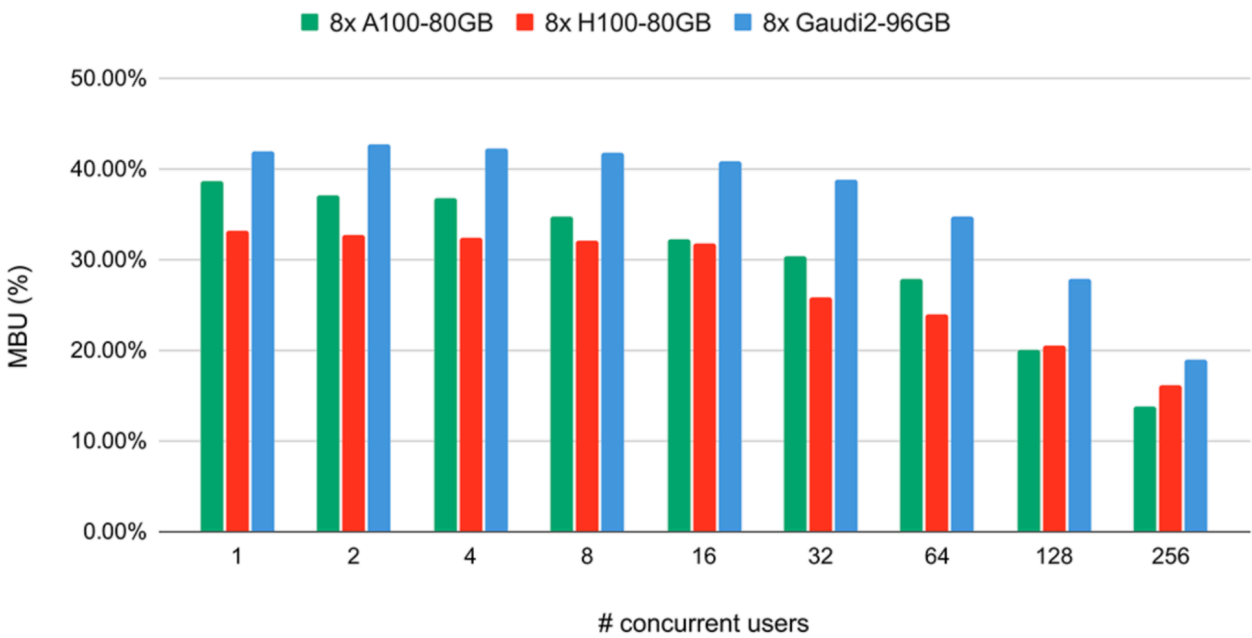
表9-1. 来自"PaLM: Scaling Language Modeling with Pathways"(Chowdhery等, 2022)的MFU示例。

模型	参数数量(十亿)	加速器芯片	模型FLOP/s利用率
GPT-3	175B	V100	21.3%
Gopher	280B	4096 TPU v3	32.5%
Megatron-Turing NLG	530B	2240 A100	30.2%
PaLM	540B	6144 TPU v4	46.2%

图9-5显示了在不同硬件上使用Llama 2-70B在FP16中进行推理过程的MBU。随着用户数量的增加, MBU下降的可能原因是每秒更高的计算负载, 使工作负载从带宽受限转变为计算受限。

Llama2-70B: Model Bandwidth Utilization

Higher is better



【图9-5. Llama 2-70B在FP16中在三个不同芯片上的带宽利用率显示, 随着并发用户数量的增加, MBU下降。图片来自"LLM Training and Inference with Intel Gaudi 2 AI Accelerators"(Databricks , 2024)。】

利用率指标有助于跟踪系统的效率。在相同硬件上类似工作负载的更高利用率通常意味着你的服务正变得更有效率。然而，目标不是获得具有最高利用率的芯片。你真正关心的是如何更快、更便宜地完成工作。如果成本和延迟都增加，更高的利用率没有任何意义。

AI加速器

软件能运行多快、多便宜取决于它运行的硬件。虽然有一些优化技术适用于所有硬件，但了解硬件允许更深入的优化。本节从推理角度看硬件，但也可以应用于训练。

AI模型和硬件的发展一直是相互交织的。缺乏足够强大的计算机是20世纪70年代第一次AI寒冬的重要原因之一。

2012年对深度学习的兴趣复苏也与计算能力密切相关。AlexNet(Krizhevsky等, 2012)广受认可的原因之一是它是第一篇成功使用GPU(图形处理单元)训练神经网络的论文。在GPU之前，如果你想训练AlexNet规模的模型，你必须使用数千个CPU，就像谷歌在AlexNet发布前几个月推出的那样。与数千个CPU相比，几个GPU对博士生和研究人员来说更容易获得，这引发了深度学习研究的蓬勃发展。

什么是加速器？

加速器是为加速特定类型的计算工作负载而设计的芯片。AI加速器是为AI工作负载设计的。主要的AI加速器类型是GPU，在2020年代初AI热潮期间，最大的经济驱动力无疑是NVIDIA。

CPU和GPU之间的主要区别在于，CPU是为通用用途设计的，而GPU是为并行处理设计的：

- CPU有几个强大的核心，高端消费级机器通常最多有64个核心。虽然许多CPU核心可以有效处理多线程工作负载，但它们在需要高单线程性能的任务上表现出色，如运行操作系统、管理I/O(输入/输出)操作或处理复杂的顺序处理。
- GPU有数千个更小、功率更低的核心，优化用于可以分解为许多较小、独立计算的任务，如图形渲染和机器学习。构成大多数ML工作负载的操作是矩阵乘法，这是高度可并行化的。

虽然追求高效并行处理增强了计算能力，但它给内存设计和功耗带来了挑战。

NVIDIA GPU的成功启发了许多旨在加速AI工作负载的加速器，包括Advanced Micro Devices(AMD)的新一代GPU、Google的TPU(张量处理单元)、Intel的Habana Gaudi、Graphcore的智能处理单元(IPU)、Groq的语言处理单元(LPU)、Cerebras的晶圆级量子处理单元(QPU)，以及许多正在推出的新产品。

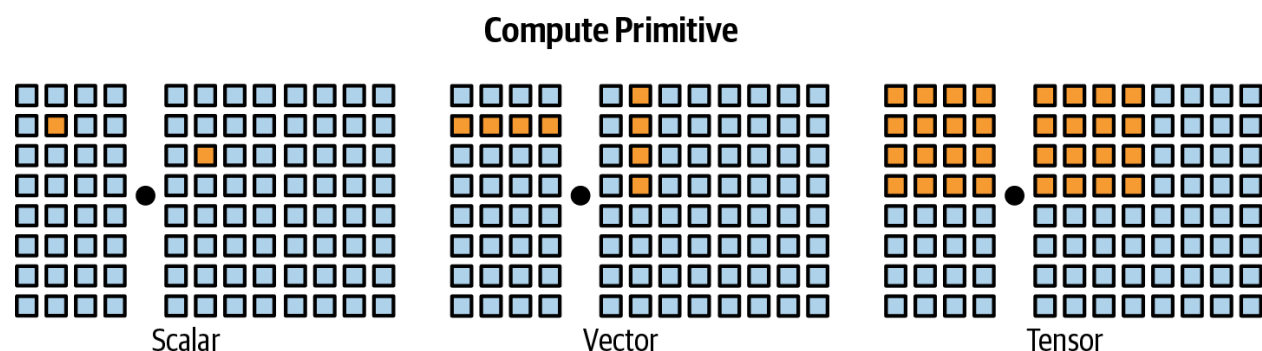
虽然许多芯片可以同时处理训练和推理，但出现的一个重要趋势是专门用于推理的芯片。Desislavov等(2023)的一项调查显示，在常用系统中，推理成本可能超过训练成本，并且在已部署的AI系统中，推理占机器学习成本的90%。

如第7章所述，由于反向传播，训练需要更多内存，并且通常更难以较低精度执行。此外，训练通常强调吞吐量，而推理则旨在最小化延迟。

因此，为推理设计的芯片通常针对更低精度和更快的内存访问进行优化，而不是大内存容量。这类芯片的例子包括Apple神经引擎、AWS Inferentia和MTIA(Meta训练与推理加速器)。为边缘计算设计的芯片，如Google的Edge TPU和NVIDIA Jetson Xavier，也通常面向推理。

还有一些为不同模型架构专门设计的芯片，如专门针对transformer的芯片。许多芯片是为数据中心设计的，越来越多的芯片是为消费设备(如手机和笔记本电脑)设计的。

不同的硬件架构有不同的内存布局 and 随时间演变的专用计算单元。这些单元针对特定数据类型进行了优化，如标量、向量或张量，如图9-6所示。



【图9-6. 不同的计算原语。图片灵感来自Chen等(2018)。】

一个芯片可能混合了为各种数据类型优化的不同计算单元。例如，GPU传统上支持向量操作，但许多现代GPU现在包含针对矩阵和张量计算优化的张量核心。而TPU则是以张量操作为主要计算原语设计的。要在硬件架构上高效运行模型，需要考虑其内存布局和计算原语。

芯片规格包含许多对评估特定用例有用的详细信息。然而，跨用例重要的主要特性是计算能力、内存大小和带宽以及功耗。我将使用GPU作为例子来说明这些特性。

计算能力

计算能力通常通过芯片在给定时间内能执行的操作数量来衡量。最常见的指标是FLOP/s，通常写作FLOPS，衡量每秒浮点操作的峰值数量。然而，在现实中，应用程序几乎不可能达到这个峰值FLOP/s。实际FLOP/s与理论FLOP/s之间的比率是一个利用率指标。

芯片在一秒内能执行的操作数量取决于数值精度——精度越高，芯片能执行的操作越少。想想如何加两个32位数字通常需要加两个16位数字的两倍计算量。由于不同芯片的优化，32位操作数量与16位操作数量的比例并不完全是一半。要了解数值精度的概述，请回顾"数值表示"部分。

表9-2显示了NVIDIA H100 SXM芯片在不同精度格式下的FLOP/s规格。

表9-2. NVIDIA H100 SXM芯片的FLOP/s规格。

数值精度	万亿FLOP/s(含稀疏性)
TF32张量核心	989
BFLOAT16张量核心	1,979
FP16张量核心	1,979
FP8张量核心	3,958

a 回顾第7章，TF32是19位而非32位格式。

内存大小和带宽

因为GPU有许多并行工作的核心，数据通常需要从内存移动到这些核心，因此数据传输速度很重要。当使用涉及大型权重矩阵和训练数据的AI模型时，数据传输至关重要。这些大量数据需要快速移动以保持核心高效运行。因此，GPU内存需要比CPU内存有更高的带宽和更低的延迟，这要求更先进的内存技术。这是使GPU内存比CPU内存更昂贵的因素之一。

具体来说，CPU通常使用DDR SDRAM(双倍数据速率同步动态随机存取存储器)，具有2D结构。GPU，尤其是高端GPU，通常使用HBM(高带宽内存)，具有3D堆叠结构。

加速器的内存通过其大小和带宽来衡量。这些数字需要在加速器所属的系统中进行评估。加速器，如GPU，通常与三级内存交互，如图9-7所示：

CPU内存 (DRAM) 加速器通常与CPU一起部署, 使它们能够访问CPU内存 (也称为系统内存、主机内存或仅CPU DRAM)。

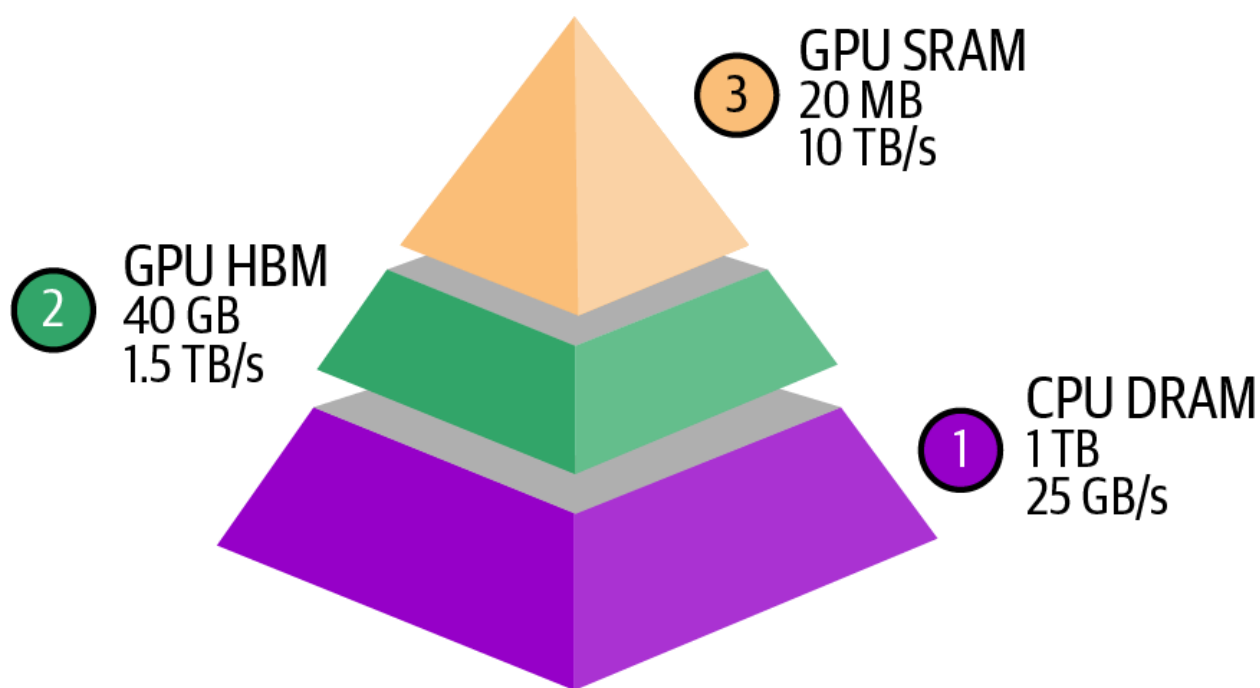
CPU内存存在这些内存类型中通常具有最低带宽, 数据传输速度范围从25 GB/s到50 GB/s。CPU内存大小各不相同。普通笔记本电脑可能有约16-64 GB, 而高端工作站可能有1 TB或更多。

GPU高带宽内存 (HBM) 这是专用于GPU的内存, 位于靠近GPU的位置, 以便比CPU内存更快地访问。

HBM提供显著更高的带宽, 数据传输速度通常从256 GB/s到超过1.5 TB/s。这种速度对于高效处理大数据传输和高吞吐量任务至关重要。消费级GPU通常有约24-80 GB的HBM。

GPU片上SRAM 直接集成到芯片中, 这种内存用于存储频繁访问的数据和指令, 以实现几乎即时的访问。它包括由SRAM制成的L1和L2缓存, 在某些架构中, 还有L3缓存。这些缓存是更广泛的片上内存的一部分, 该内存还包括寄存器文件和共享内存等其他组件。

RAM具有极高的数据传输速度, 通常超过10 TB/s。GPU SRAM的大小很小, 通常为40 MB或更小。



【图9-7. AI加速器的内存层次结构。数字仅供参考。实际数字因芯片而异。】

大量GPU优化是关于如何充分利用这种内存层次结构。然而, 截至撰写本文时, PyTorch和TensorFlow等流行框架尚未允许对内存访问进行精细控制。这导致许多AI研究人员和工程师对GPU编程语言感兴趣, 如CUDA (最初为计算统一设备架构)、OpenAI的Triton和ROCm (Radeon开放计算)。后者是AMD对NVIDIA专有CUDA的开源替代方案。

功耗

芯片依靠晶体管执行计算。每次计算都是通过晶体管的开关完成的，这需要能量。一个GPU可以拥有数十亿晶体管——NVIDIA A100有540亿晶体管，而NVIDIA H100有800亿晶体管。当加速器被高效使用时，数十亿晶体管迅速切换状态，消耗大量能源并产生大量热量。这种热量需要冷却系统，这些系统也消耗电力，增加了数据中心的总体能源消耗。

芯片能源消耗可能对环境产生巨大影响，增加了公司投资绿色数据中心技术的压力。一个NVIDIA H100以峰值运行一年消耗约7,000千瓦时。相比之下，美国普通家庭的年度电力消耗为10,000千瓦时。这就是为什么电力是扩大计算的瓶颈。

加速器通常通过最大功耗或TDP(热设计功率)代理指标指定其功耗：

- 最大功耗表示芯片在满负荷下可能的峰值功耗。
- TDP表示当芯片在典型工作负载下运行时，冷却系统需要散热的最大热量。虽然它不是功耗的精确衡量，但它表明了预期的功耗。对于CPU和GPU，最大功耗可能是TDP的1.1到1.5倍，尽管确切的关系因特定架构和工作负载而异。

如果你选择云提供商，你不需要担心冷却或电力。然而，这些数字仍然有助于了解加速器对环境的影响和总体电力需求。

选择加速器

使用什么加速器取决于你的工作负载。如果你的工作负载是计算受限的，你可能想寻找具有更多FLOP/s的芯片。如果你的工作负载是内存受限的，为具有更高带宽和更多内存的芯片支付费用将使你的生活更轻松。

评估购买哪些芯片时，有三个主要问题：

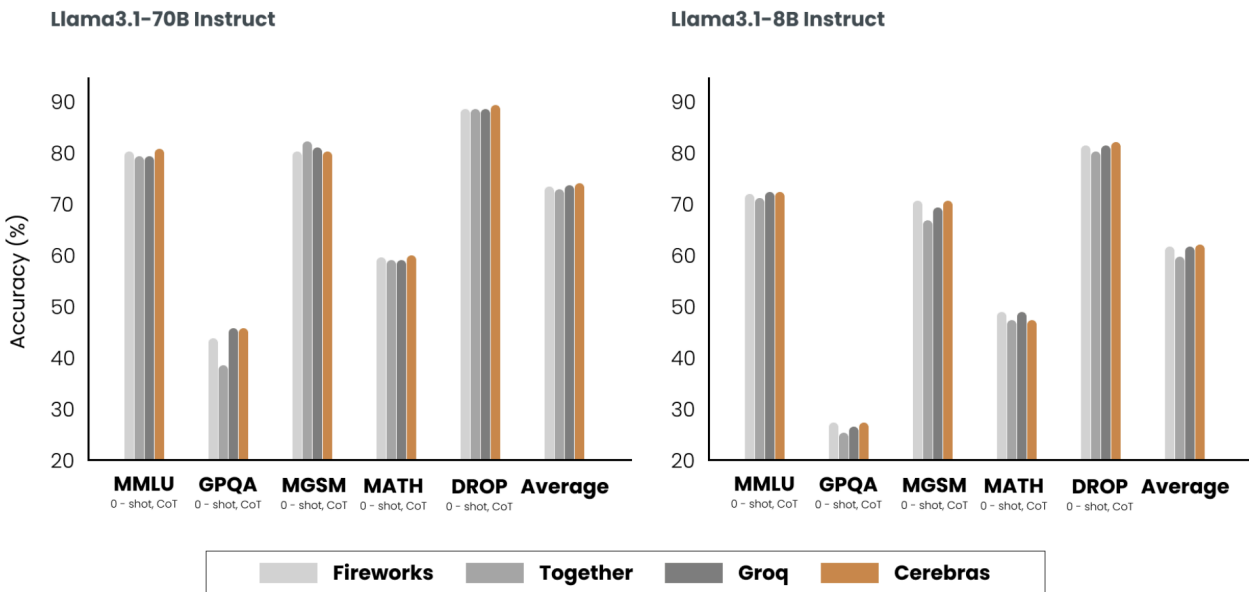
1. 硬件能否运行你的工作负载？
2. 需要多长时间才能做到？
3. 成本是多少？

FLOP/s、内存大小和内存带宽是帮助你回答前两个问题的三个重要数字。最后一个问题很简单。云提供商的定价通常基于使用量，各提供商之间相当相似。如果你购买自己的硬件，成本可以根据初始价格和持续功耗计算。

推理优化

推理优化可以在模型、硬件或服务层面进行。为了说明它们的区别，考虑射箭。模型层面的优化就像制作更好的箭。硬件层面的优化就像训练更强大更好的射手。服务层面的优化就像改进整个射击过程，包括弓和瞄准条件。

理想情况下，为速度和成本优化模型不应改变模型的质量。然而，许多技术可能导致模型性能下降。图9-8显示了不同推理服务提供商提供的相同Llama模型在不同基准测试上的性能。



【图9-8. 推理服务提供商可能使用改变模型行为的优化技术，导致不同提供商的模型质量略有不同。实验由Cerebras(2024)进行。】

由于硬件设计超出了本书的范围，我将讨论模型和服务层面的技术。虽然这些技术是分开讨论的，但请记住，在生产中，优化通常涉及多个层面的技术。

模型优化

模型层面的优化旨在使模型更高效，通常通过修改模型本身，这可能改变其行为。截至撰写本文时，许多基础模型遵循transformer架构并包含自回归语言模型组件。这些模型有三个特性使推理资源密集：模型大小、自回归解码和注意力机制。让我们讨论解决这些挑战的方法。

模型压缩

模型压缩涉及减小模型大小的技术。使模型更小也可以使其更快。本书已经讨论了两种模型压缩技术：量化和蒸馏。量化(在第7章中讨论)是减少模型精度以减小其内存占用并增加其吞吐量。模型蒸馏(在第8章中讨论)是训练一个小模型来模仿大模型的行为。

模型蒸馏表明，使用更少的参数可以捕获大模型的行为。在大模型内，是否存在能够捕获整个模型行为的参数子集？这是剪枝的核心概念。

神经网络上下文中的剪枝有两个含义。一个是移除神经网络的整个节点，这意味着改变其架构并减少其参数数量。另一个是找到对预测最不有用的参数并将其设为零。在这种情况下，剪枝不会减少总参数数量，只会减少非零参数的数量。这使模型更稀疏，既减少了模型的存储空间，又加速了计算。

剪枝后的模型可以直接使用，也可以进一步微调以调整剩余参数并恢复剪枝过程可能导致的任何性能下降。剪枝可以帮助发现有前途的模型架构(Liu等, 2018)。这些被剪枝的架构，比剪枝前的架构更小，也可以从头开始训练(Zhu等, 2017)。

在文献中，有许多令人鼓舞的剪枝结果。例如，Frankle和Carbin(2019)显示，剪枝技术可以减少某些训练网络中90%以上的非零参数数量，减少内存占用并提高速度，而不影响准确性。然而，在实践中，截至撰写本文时，剪枝并不常见。它更难实现，因为它需要了解原始模型的架构，并且它带来的性能提升通常远低于其他方法。剪枝还会导致稀疏模型，并非所有硬件架构都设计为利用所产生的稀疏性。

权重量化是迄今为止最流行的方法，因为它易于使用，对许多模型都能开箱即用，并且非常有效。将模型的精度从32位减少到16位会将其内存占用减少一半。然而，我们接近量化的极限——我们不能低于每个值1位。蒸馏也很常见，因为它可以产生一个较小的模型，其行为与更大的模型相当，满足你的需求。

克服自回归解码瓶颈

如第2章所述，自回归语言模型一个接一个地生成标记。如果生成一个标记需要100毫秒，100个标记的响应将需要10秒。这个过程不仅慢，还很昂贵。在模型API提供商中，一个输出标记的成本大约是输入标记的二到四倍。在一项实验中，Anyscale发现单个输出标记对延迟的影响可能与100个输入标记相同(Kadous等, 2023)。通过少量百分比改进自回归生成过程可以显著改善用户体验。

随着该领域的快速发展，新技术正在开发中，以克服这个看似不可能的瓶颈。也许有一天，会有不存在这个瓶颈的架构。这里介绍的技术是为了说明解决方案可能是什么样子，但技术仍在发展中。

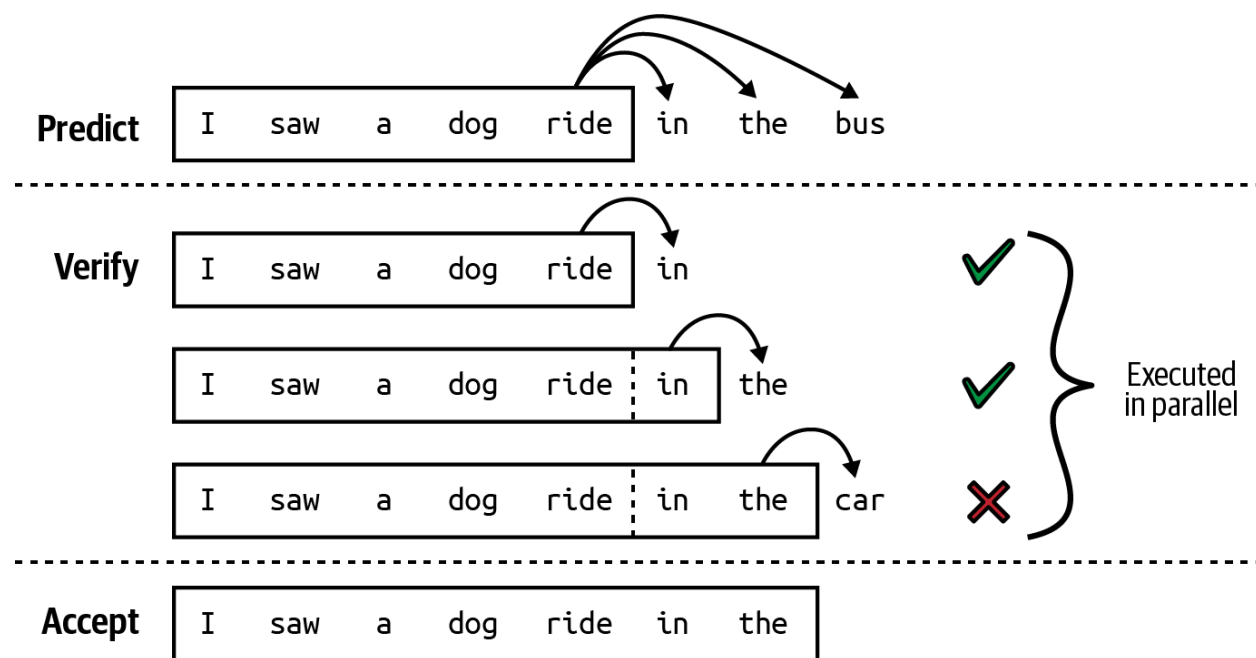
推测解码 推测解码(也称为推测采样)使用更快但功能较弱的模型生成一系列标记，然后由目标模型验证。目标模型是你想要使用的模型。更快的模型称为草稿或提案模型，因为它提出了输出草稿。

假设输入标记是 $x_1, x_2, \dots, x_t$:

1. 草稿模型生成K个标记的序列: $x_{t+1}, x_{t+2}, \dots, x_{t+K}$ 。
2. 目标模型并行验证这K个生成的标记。
3. 目标模型接受草稿标记中从左到右最长的子序列，这些是目标模型同意使用的标记。
4. 假设目标模型接受j个草稿标记, $x_{t+1}, x_{t+2}, \dots, x_{t+j}$ 。目标模型然后生成一个额外的标记, x_{t+j+1} 。

5. 返回步骤1, 草稿模型生成K个标记, 基于 $x_1, x_2, \dots, x_t, x_{t+1}, x_{t+2}, \dots, x_{t+j}$ 。过程在图9-9中可视化。

如果没有草稿标记被接受, 这个循环只产生由目标模型生成的一个标记。如果所有草稿标记都被接受, 这个循环产生K+1个标记, 其中K个由草稿模型生成, 一个由目标模型生成。



【图9-9. 草稿模型生成一系列K个标记, 主模型接受它同意的最长子序列。图片来自"Blockwise Parallel Decoding for Deep Autoregressive Models"(Stern等, 2018)。】

如果所有草稿序列都被拒绝, 目标模型必须生成整个响应并验证它, 这可能导致延迟增加。然而, 由于以下三个见解, 这种情况可以避免:

1. 目标模型验证一系列标记所需的时间少于生成它所需的时间, 因为验证是可并行化的, 而生成是顺序的。推测解码有效地将解码的计算特性转变为预填充的计算特性。
2. 在输出标记序列中, 有些标记比其他标记更容易预测。可以找到一个更弱的草稿模型, 能够正确预测这些容易预测的标记, 从而获得草稿标记的高接受率。
3. 解码是内存带宽受限的, 这意味着在编码过程中, 通常有空闲的FLOP可以用于免费验证。

接受率是领域相关的。对于遵循特定结构的文本(如代码), 接受率通常更高。K值越大, 目标模型的验证调用次数越少, 但草稿标记的接受率较低。草稿模型可以是任何架构, 但理想情况下它应该与目标模型共享相同的词汇表和分词器。你可以训练自定义草稿模型或使用现有的较弱模型。

例如，为了加速Chinchilla-70B的解码过程，DeepMind训练了一个具有相同架构的4B参数草稿模型(Chen等，2023)。草稿模型生成标记的速度是目标模型的八倍(1.8毫秒/标记与14.1毫秒/标记相比)。这将整体响应延迟减少了一半以上，而不影响响应质量。对T5-XXL也实现了类似的加速(Laviathan等，2022)。

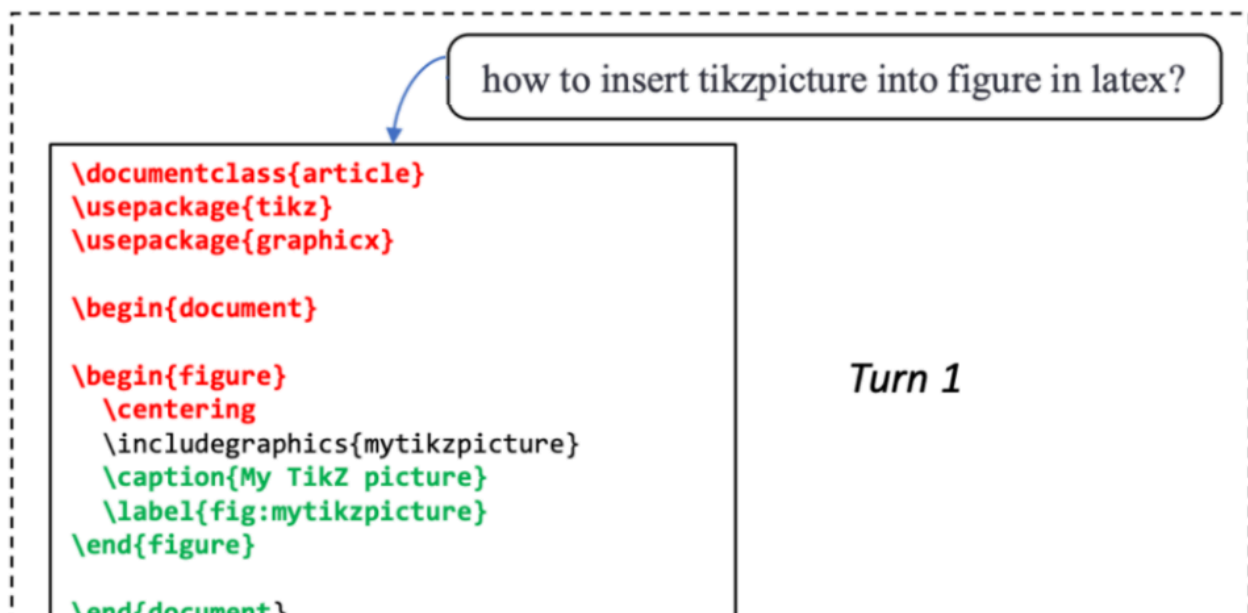
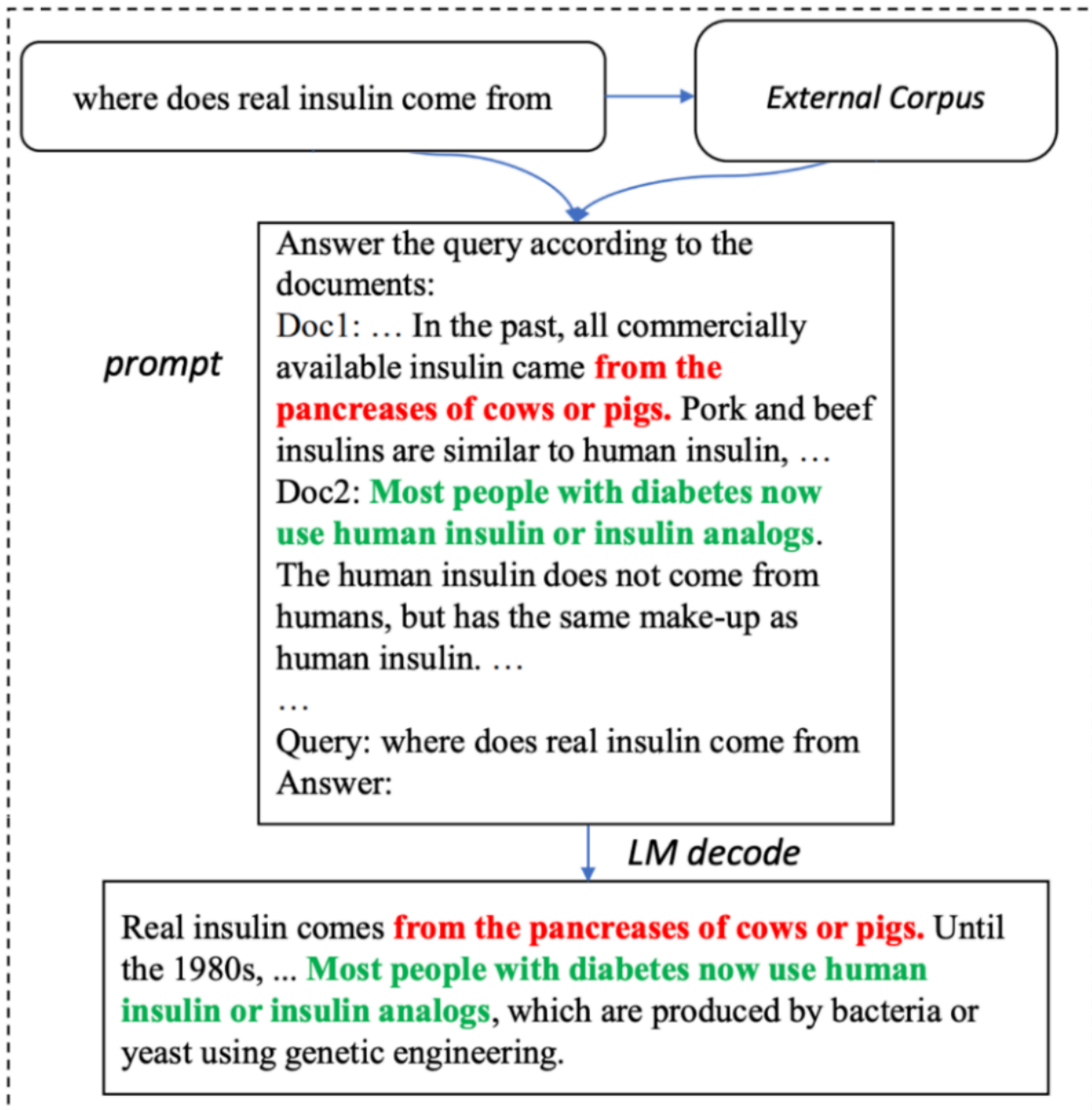
这种方法已经获得了关注，因为它相对容易实现，并且不会改变模型的质量。例如，可以在PyTorch中用50行代码实现它。它已被整合到流行的推理框架中，如vLLM、TensorRT-LLM和llama.cpp。

参考推理 通常，响应需要引用来自输入的标记。例如，如果你向模型询问关于附加文档的问题，模型可能会逐字重复文档中的一段文字。另一个例子是，如果你要求模型修复代码中的错误，模型可能会重用原始代码的大部分，只做少量更改。我们能否复制这些重复的标记以加速生成，而不是让模型生成这些重复的标记？这就是参考推理的核心思想。

参考推理类似于推测解码，但不是使用模型生成草稿标记，而是从输入中选择草稿标记。主要挑战是开发一种算法，在每个解码步骤识别上下文中最相关的文本片段。最简单的选择是找到匹配当前标记的文本片段。

与推测解码不同，参考推理不需要额外的模型。然而，它只在上下文和输出之间有显著重叠的生成场景中有效，如检索系统、编码或多轮对话。在"Inference with Reference: Lossless Acceleration of Large Language Models"(Yang等，2023)中，这种技术帮助在此类用例中实现两倍的生成速度。

参考推理工作方式的例子如图9-10所示。



【图9-10. 参考推理的两个例子。成功从输入复制的文本段以红色和绿色显示。图片来自Yang等(2023)。图片在CC BY 4.0下授权。】

并行解码 一些技术旨在打破顺序依赖性, 而不是使用草稿标记使自回归生成更快。给定现有标记序列 $x_1, x_2, \dots, x_t$, 这些技术尝试同时生成 $x_{t+1}, x_{t+2}, \dots, x_{t+k}$ 。这意味着模型在不知道前一个标记是 x_{t+1} 的情况下生成 x_{t+2} 。

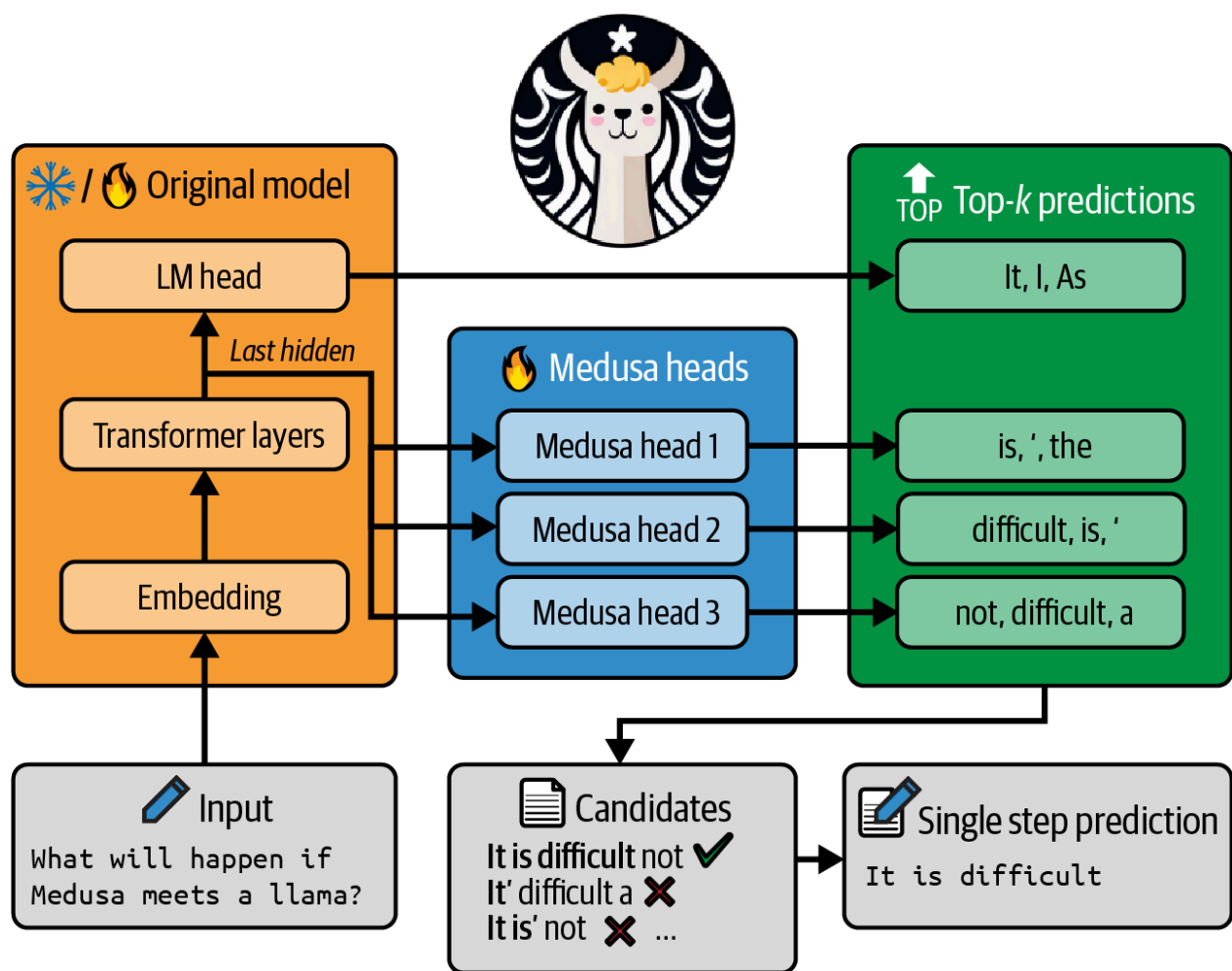
这可以工作是因为对现有序列的了解通常足以预测接下来的几个标记。例如, 给定"猫坐着", 在不知道下一个标记是"在"、"下"或"后面"的情况下, 你仍然可能预测它之后的词是"上"。

并行标记可以由同一解码器生成, 如前瞻解码(Fu等, 2024), 或由不同的解码头生成, 如美杜莎(Cai等, 2024)。在美杜莎中, 原始模型扩展了多个解码头, 每个头是一个小型神经网络层, 然后训练预测特定位置的将来标记。如果原始模型训练预测下一个标记 x_{t+1} , 第 k 个头将预测标记 x_{t+k+1} 。这些头与原始模型一起训练, 但原始模型是冻结的。NVIDIA声称美杜莎帮助提升Llama 3.1标记生成速度在HGX H200 GPU上高达1.9倍(Eassa等, 2024)。

然而, 因为这些标记不是按顺序生成的, 它们需要被验证以确保它们相互匹配。并行解码的重要部分是验证和集成。前瞻解码使用雅可比方法验证生成的标记, 其工作方式如下:

1. 并行生成 K 个未来标记。
2. 验证这 K 个标记与上下文的一致性和连贯性。
3. 如果一个或多个标记验证失败, 模型不会聚合所有 K 个未来标记, 而是只重新生成或调整这些失败的标记。
4. 模型继续完善生成的标记, 直到它们全部通过验证并集成到最终输出中。这种并行解码算法系列也称为雅可比解码。

另一方面, 美杜莎使用基于树的注意力机制来验证和集成标记。每个美杜莎头为每个位置产生几个选项。然后将这些选项组织成树状结构, 以选择最有前途的组合。该过程如图9-11所示。



【图9-11. 在美杜莎(Cai等, 2024)中, 每个头为一个标记位置预测几个选项。从这些选项中选择最有前途的序列。图片改编自论文, 该论文在CC BY 4.0下授权。】

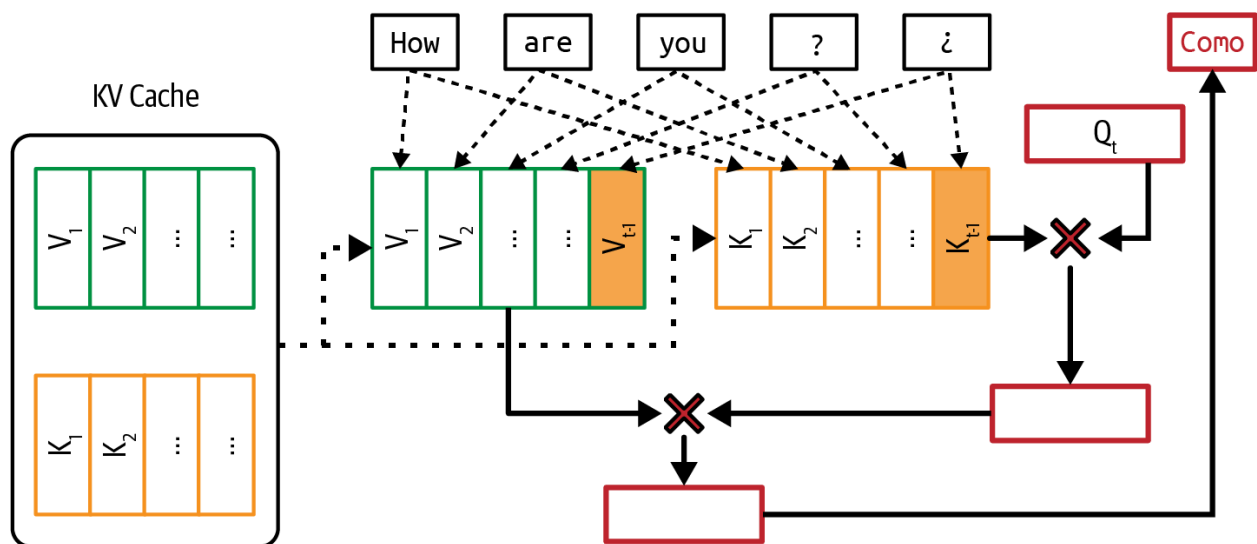
虽然能够规避顺序依赖的前景令人吸引, 但并行解码并不直观, 一些技术, 如美杜莎, 可能难以实现。

注意力机制优化

回顾第2章, 生成下一个标记需要所有前一个标记的键和值向量。这意味着以下适用:

- 生成标记 x_t 需要标记 $x_1, x_2, \dots, x_{t-1}$ 的键和值向量。
- 生成标记 x_{t+1} 需要标记 $x_1, x_2, \dots, x_{t-1}, x_t$ 的键和值向量。

当生成标记 x_{t+1} 时, 不是重新计算标记 $x_1, x_2, \dots, x_{t-1}$ 的键和值向量, 而是重用这些来自前一步的向量。这意味着你只需要计算最近标记 x_t 的键和值向量。存储键和值向量以重用的缓存称为KV缓存。新计算的键和值向量然后添加到KV缓存中, 如图9-12所示。



【图9-12. 为避免在每个解码步骤重新计算键和值向量，使用KV缓存存储这些向量以重用。】

注意 KV缓存仅在推理过程中使用，不在训练中使用。在训练期间，因为序列中的所有标记都是预先知道的，所以下一个标记生成可以一次性计算，而不是像推理期间那样顺序计算。因此，不需要KV缓存。

因为生成一个标记需要计算与所有前一个标记的注意力分数，所以随着序列长度的增加，注意力计算的数量呈指数增长。另一方面，KV缓存大小随序列长度线性增长。

KV缓存大小也随着批处理大小的增加而增长。Google的一篇论文计算，对于具有多头注意力的500B+模型，批处理大小512，上下文长度2048，KV缓存总计3TB(Pope等，2022)。这是该模型权重大小的三倍。

KV缓存大小最终受限于可用的硬件存储，为运行长上下文应用程序创造了瓶颈。大缓存大小也需要时间加载到内存中，这对严格要求延迟的应用程序可能是个问题。

注意力机制的计算和内存要求是很难拥有更长上下文的原因之一。

许多技术已被开发来使注意力机制更高效。总体而言，它们分为三类：重新设计注意力机制、优化KV缓存以及为注意力计算编写内核。

计算KV缓存大小

在没有任何优化的情况下，KV缓存所需的内存计算如下：

$$2 \times B \times S \times L \times H \times M$$

其中：

- B: 批处理大小
- S: 序列长度
- L: transformer层数
- H: 模型维度
- M: 缓存数值表示所需的内存(如FP16或FP32)

随着上下文长度的增加, 这个值可能变得相当大。例如, LLama 2 13B有40层, 模型维度为5,120。批处理大小为32, 序列长度为2,048, 每个值2字节, 其KV缓存所需的内存, 在没有任何优化的情况下, 是 $2 \times 32 \times 2,048 \times 40 \times 5,120 \times 2 = 54 \text{ GB}$ 。

重新设计注意力机制

这些技术涉及改变注意力机制的工作方式。尽管这些技术有助于优化推理, 但因为它们直接改变模型架构, 所以只能在训练或微调期间应用。

例如, 在生成新标记时, 局部窗口注意力不是关注所有先前标记, 而是只关注固定大小窗口内的附近标记(Beltagy等, 2020)。这将有效序列长度减少到固定大小窗口, 减少了KV缓存和注意力计算。如果平均序列长度为10,000个标记, 关注1,000个标记的窗口大小将使KV缓存大小减少10倍。

局部窗口注意力可以与全局注意力交错使用, 局部注意力捕获附近上下文; 全局注意力捕获文档中的任务特定信息。

跨层注意力(Brandon等, 2024)和多查询注意力(Shazeer, 2019)都通过减少键值对的数量来减少KV缓存的内存占用。跨层注意力在相邻层之间共享键和值向量。三层共享相同的键值向量意味着将KV缓存减少三倍。另一方面, 多查询注意力在查询头之间共享键值向量。

分组查询注意力(Ainslie等, 2023)是多查询注意力的泛化。它不是为所有查询头使用一组键值对, 而是将查询头放入更小的组中, 只在同一组中的查询头之间共享键值对。这允许在查询头数量和键值对数量之间更灵活的平衡。

Character.AI是一款AI聊天机器人应用程序, 它分享说他们的平均对话历史包含180条消息(2024)。鉴于通常较长的序列, 推理吞吐量的主要瓶颈是KV缓存大小。三种注意力机制设计——多查询注意力、交错局部注意力和全局注意力以及跨层注意力——帮助他们将KV缓存减少了20多倍。更重要的是, 这种显著的KV缓存减少意味着对他们来说, 内存不再是服务大批量大小的瓶颈。

优化KV缓存大小

管理KV缓存的方式对于缓解推理期间的内存瓶颈和实现更大的批处理大小至关重要, 特别是对于长上下文应用程序。许多技术正在积极开发中, 以减少和管理KV缓存。

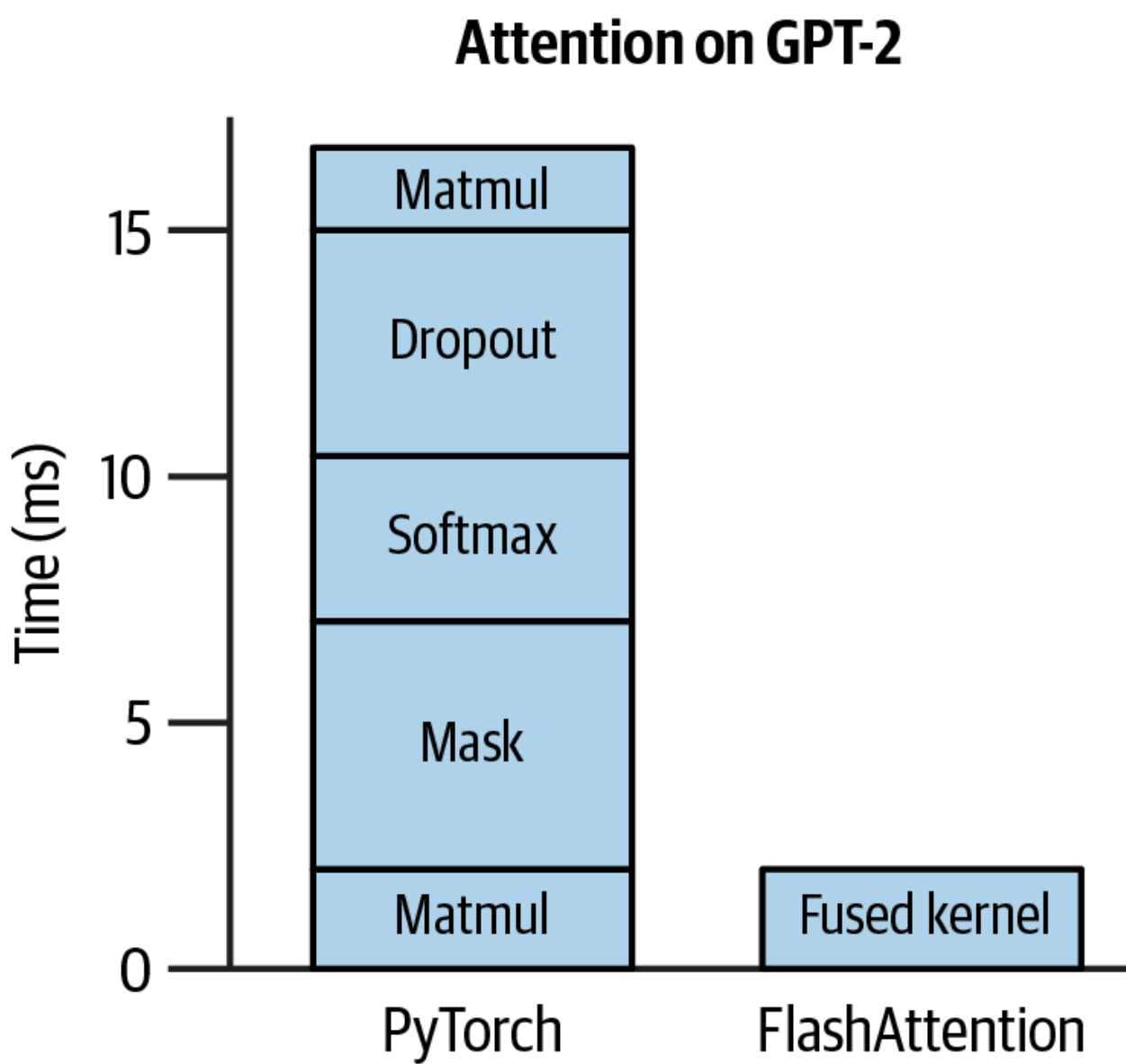
最快速发展的推理框架之一vLLM因引入PagedAttention而广受欢迎, 该技术通过将KV缓存分成非连续块来优化内存管理, 减少碎片化, 并实现灵活的内存共享, 提高LLM服务效率(Kwon等, 2023)。

其他技术包括KV缓存量化 (Hooper等, 2024; Kang等, 2024)、自适应KV缓存压缩 (Ge等, 2023) 和选择性KV缓存 (Liu等, 2024)。

为注意力计算编写内核

这种方法不是改变机制设计或优化存储，而是研究如何计算注意力分数并找到使这种计算更高效的方法。当考虑执行计算的硬件时，这种方法最有效。为特定芯片优化的代码称为内核。下一节将进一步讨论内核编写。

最著名的为注意力计算优化的内核之一是FlashAttention (Dao等, 2022)。这个内核将transformer模型中常用的许多操作融合在一起，使它们运行得更快，如图9-13所示。



【图9-13. FlashAttention是将几个常见运算符融合在一起的内核。改编自在BSD 3-Clause许可下的原始图像。】

内核和编译器

内核是为特定硬件加速器(如GPU或TPU)优化的专业代码片段。它们通常被编写用于执行需要重复执行的计算密集型例程,通常是并行执行,以最大化这些加速器的性能。

常见的AI操作,包括矩阵乘法、注意力计算和卷积操作,都有专门的内核,使它们在不同硬件上的计算更高效。

编写内核需要深入了解底层硬件架构。这包括了解内存层次结构的构建方式(如缓存、全局内存、共享内存和寄存器)以及数据如何在这些不同级别之间访问和移动。

此外,内核通常用较低级别的编程语言编写,如CUDA(用于NVIDIA GPU)、Triton(OpenAI开发的用于编写自定义内核的语言)和ROCm(用于AMD GPU)。这些语言允许对线程管理和内存访问进行精细控制,但也比大多数AI工程师熟悉的语言(如Python)更难学习。

由于这种进入障碍,编写内核曾经是少数人实践的黑魔法。像NVIDIA和AMD这样的芯片制造商雇用优化工程师编写内核,使他们的硬件对AI工作负载高效,而PyTorch和TensorFlow等AI框架则雇用内核工程师在不同加速器上优化他们的框架。

然而,随着推理优化需求的增加和加速器的普及,更多AI工程师对编写内核产生了兴趣。有许多很好的在线内核编写教程。在这里,我将介绍四种常用于加速计算的常见技术:

向量化 给定循环或嵌套循环,不是一次处理一个数据元素,而是同时执行内存中连续的多个数据元素。这通过最小化数据I/O操作来减少延迟。

并行化 将输入数组(或n维数组)分成可以在不同核心或线程上同时处理的独立块,加速计算。

循环平铺 针对硬件的内存布局和缓存优化循环中的数据访问顺序。这种优化依赖于硬件。高效的CPU平铺模式可能在GPU上效果不佳。

操作符融合 将多个操作符组合成单次传递,以避免冗余内存访问。例如,如果两个循环在同一数组上操作,它们可以融合成一个,减少数据读写次数。

虽然向量化、并行化和循环平铺可以广泛应用于不同模型,但操作符融合需要更深入了解模型的特定操作符和架构。因此,操作符融合需要优化工程师更多的关注。

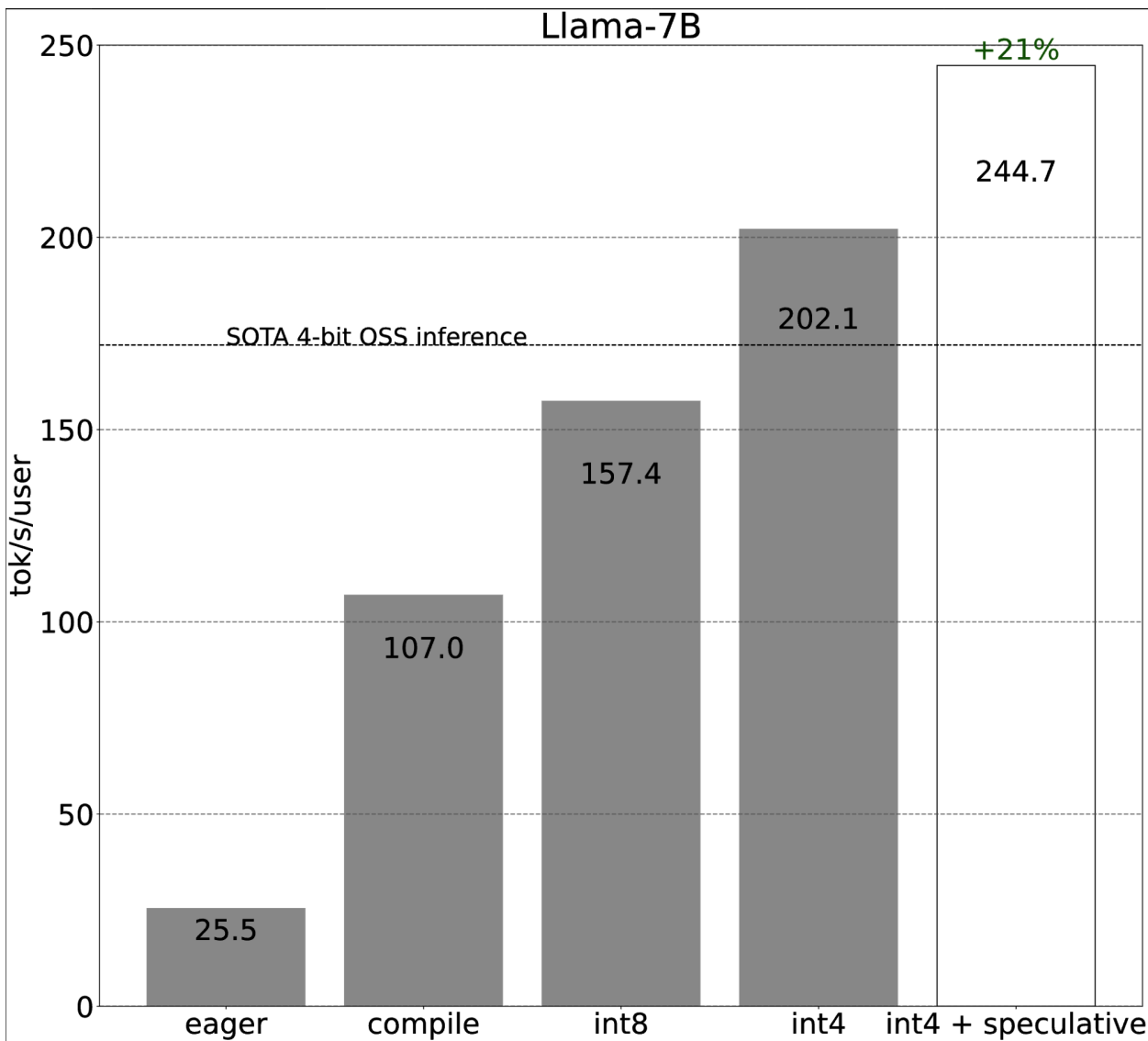
内核针对硬件架构进行优化。这意味着每当引入新的硬件架构时,都需要开发新的内核。例如,FlashAttention(Dao等, 2022)最初主要为NVIDIA A100 GPU开发。后来,为H100 GPU引入了FlashAttention-3(Shah等, 2024)。

模型脚本指定一系列需要执行的操作以执行该模型。要在GPU等硬件上运行此代码，必须将其转换为与该硬件兼容的语言。这个过程称为降级。将代码降级以在特定硬件上运行的工具称为编译器。编译器连接ML模型和运行它们的硬件。在降级过程中，这些操作尽可能转换为专用内核，以在目标硬件上更快运行。

PyTorch的推理优化案例研究

图9-14显示了PyTorch团队通过以下优化步骤为Llama-7B提供多少吞吐量改进(PyTorch, 2023)：

- 1. 调用torch.compile将模型编译成更高效的内核。
- 2. 将模型权重量化为INT8。
- 3. 进一步将模型权重量化为INT4。
- 4. 添加推测解码。



【图9-14. PyTorch中不同优化技术的吞吐量改进。图片来自PyTorch(2023)。】

实验在具有80 GB内存的A100 GPU上运行。这些优化步骤如何影响模型的输出质量尚不清楚。

编译器可以是独立工具，如Apache TVM和MLIR(多级中间表示)，或集成到ML和推理框架中，如torch.compile(PyTorch中的功能)、XLA(加速线性代数，最初由TensorFlow开发，开源版本称为OpenXLA)以及内置于TensorRT中的编译器，该编译器针对NVIDIA GPU进行了优化。AI公司可能有自己的编译器，其专有内核设计用于加速自己的工作负载。

推理服务优化

大多数服务级优化技术侧重于资源管理。给定固定数量的资源(计算和内存)和动态工作负载(来自用户的可能涉及不同模型的推理请求)，目标是有效地分配资源到这些工作负载，以优化延迟和成本。与许多模型级技术不同，服务级技术不修改模型，不应改变输出质量。

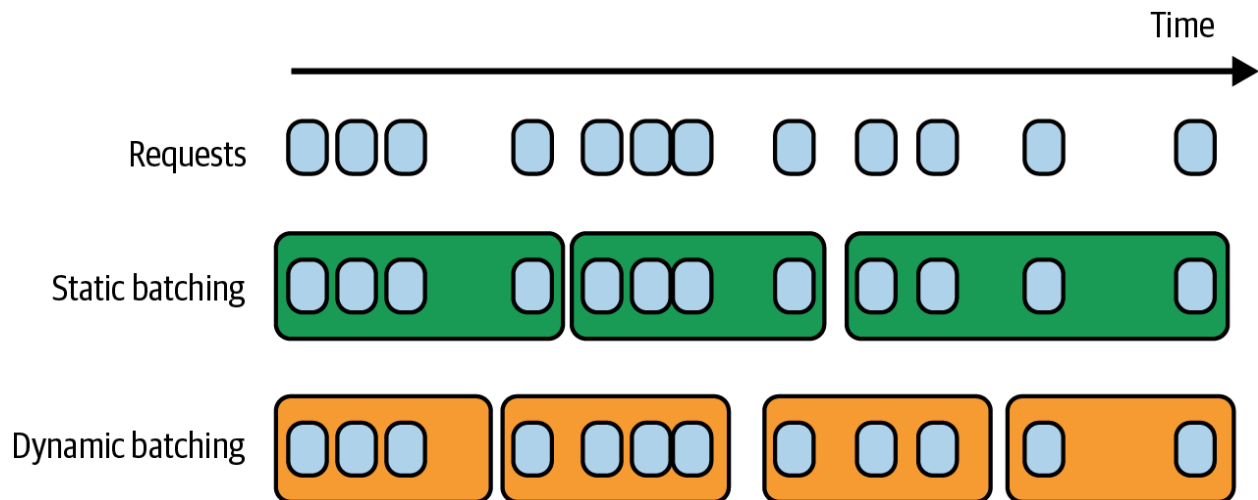
批处理

降低成本最简单的方法之一是批处理。在生产中，你的推理服务可能同时接收多个请求。批处理是将大约同时到达的请求放在一起处理，而不是单独处理每个请求，这可以显著降低服务的吞吐量。如果单独处理每个请求就像每个人开自己的车，那么批处理就像把他们放在一辆公共汽车上。公共汽车可以移动更多人，但也可能使每个人的旅程更长。然而，如果你智能地做到这一点，对延迟的影响可以最小化。

批处理的三种主要技术是：静态批处理、动态批处理和连续批处理。

最简单的批处理技术是静态批处理。服务将固定数量的输入分组到一个批次中。就像等到每个座位都满了才出发的公共汽车。静态批处理的缺点是所有请求都必须等到批次填满才能执行。因此，批次中的第一个请求会延迟到最后一个请求到达，无论最后一个请求多晚。

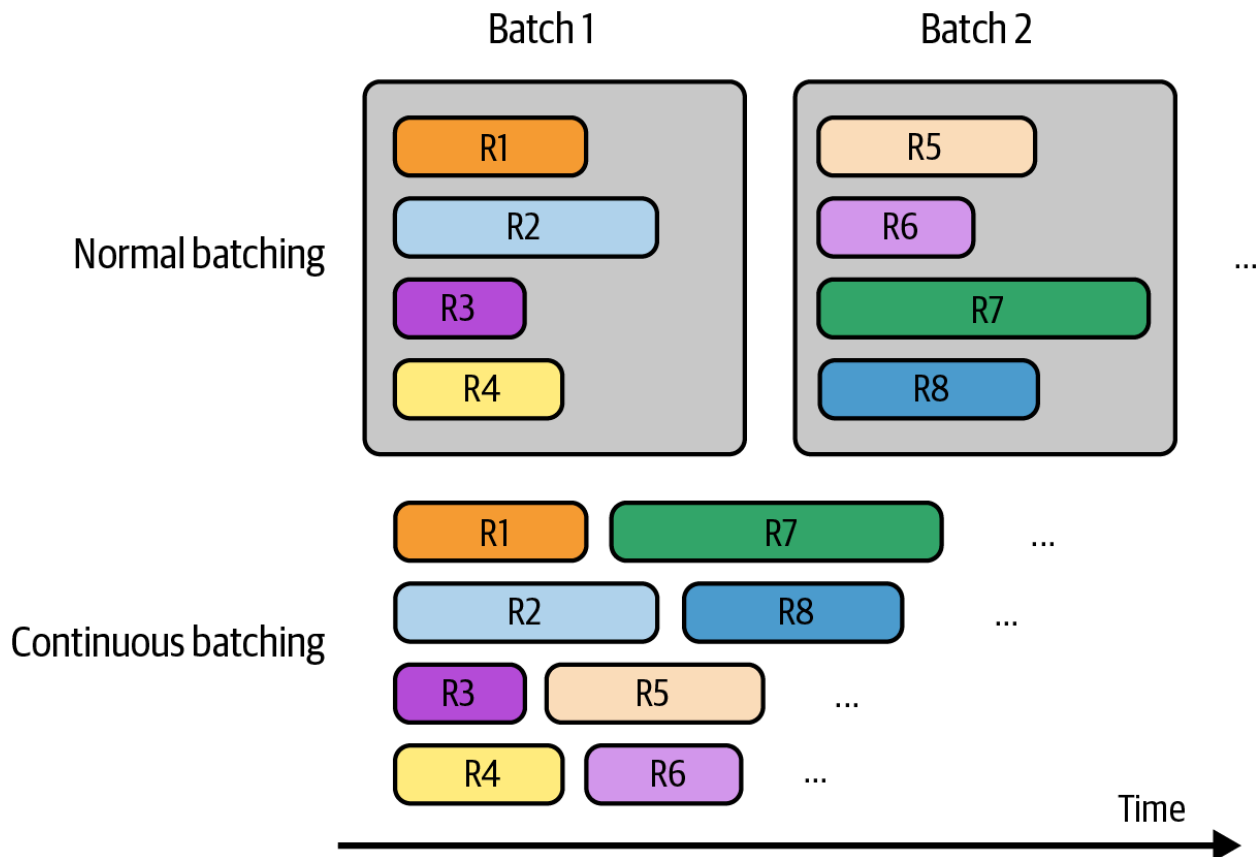
另一方面，动态批处理为每个批次设置了最大时间窗口。如果批处理大小为4，窗口为100毫秒，服务器会在有4个请求或100毫秒过去时处理批次，以先发生者为准。就像按固定时间表或满员时离开的公共汽车。这种方法控制了延迟，使较早的请求不会被较晚的请求延迟。缺点是处理时批次可能不会总是满的，可能导致计算浪费。静态批处理和动态批处理如图9-15所示。



【图9-15. 动态批处理保持延迟可管理，但可能计算效率较低。】

在简单的批处理实现中，所有批处理请求必须在返回其响应之前完成。对于LLM，一些请求可能比其他请求花费更长时间。如果批次中一个请求生成只有10个响应标记，而另一个请求生成1,000个响应标记，那么短响应必须等到长响应完成才能返回给用户。这会导致短请求不必要的延迟。

连续批处理允许批次中的响应在完成后立即返回给用户。它的工作原理是选择性地批处理不会导致一个响应的生成延迟另一个响应的操作，如Orca论文(Yu等, 2022)中引入的。批次中的请求完成并返回其响应后，服务可以将另一个请求添加到批次中取代它，使批处理连续。就像在放下一名乘客后可以立即接另一名乘客以最大化占用率的公共汽车。连续批处理，也称为飞行中批处理，如图9-16所示。



【图9-16. 使用连续批处理，完成的响应可以立即返回给用户，新请求可以替代它们处理。】

解耦预填充和解码

LLM推理包括两个步骤：预填充和解码。因为预填充是计算受限的，而解码是内存带宽受限的，使用同一台机器执行两者可能导致它们低效地争夺资源，显著减慢TTFT和TPOT。想象一个GPU已经接近其峰值计算能力处理预填充和解码。它可能能够处理另一个低计算任务，如解码。然而，向这个GPU添加一个新查询意味着引入一个预填充作业和一个解码作业。这一个预填充作业可能耗尽现有解码作业的计算资源，减慢这些请求的TPOT。

推理服务器的一种常见优化技术是分解预填充和解码。“DistServe”(Zhong等, 2024)和“Inference Without Interference”(Hu等, 2024)表明，对于各种流行的LLM和应用，将预填充和解码操作分配给不同的实例(如不同的GPU)可以显著提高处理请求的数量，同时遵守延迟要求。尽管解耦需要将中间状态从预填充实例传输到解码实例，但论文表明，在具有高带宽连接(如节点内的NVLink)的现代GPU集群中，通信开销并不大。

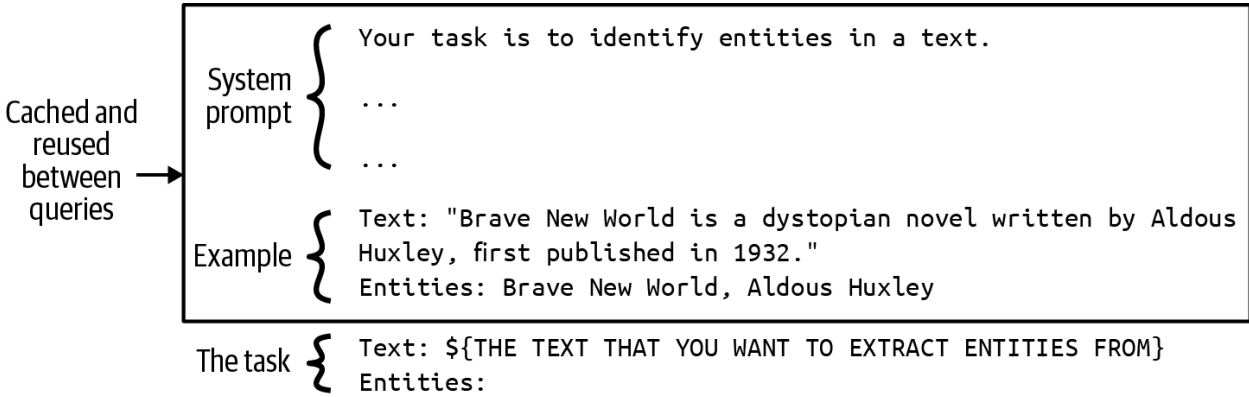
预填充实例与解码实例的比例取决于许多因素，如工作负载特性(例如，更长的输入长度需要更多的预填充计算)和延迟要求(例如，你是否想要更低的TTFT或TPOT)。例如，如果输入序列通常很长，并且你想优先考虑TTFT，这个比例可以在2:1到4:1之间。如果输入序列短，你想优先考虑TPOT，这个比例可以是1:2到1:1。

提示缓存

应用程序中的许多提示有重叠的文本段。提示缓存存储这些重叠段以重用，所以你只需要处理它们一次。不同提示中常见的重叠文本段是系统提示。没有提示缓存，你的模型需要处理每个查询的系统提示。有了提示缓存，系统提示只需要为第一个查询处理一次。

提示缓存对涉及长文档的查询很有用。例如，如果你的许多用户查询与同一个长文档（如书籍或代码库）有关，这个长文档可以被缓存并在查询之间重用。它对长对话也很有用，早期消息的处理可以被缓存并在预测未来消息时重用。

提示缓存如图9-17所示。它也称为上下文缓存或前缀缓存。



【图9-17. 使用提示缓存，不同提示中的重叠段可以被缓存和重用。】

对于具有长系统提示的应用程序，提示缓存可以显著减少延迟和成本。如果你的系统提示是1,000个标记，并且你的应用程序每天生成一百万个模型API调用，提示缓存将使你每天节省处理约十亿个重复输入标记！然而，这并不完全免费。像KV缓存一样，提示缓存大小可能相当大并占用内存空间。除非你使用具有此功能的模型API，否则实现提示缓存可能需要大量工程努力。

自2023年11月Gim等人引入以来，提示缓存已被迅速纳入模型API。截至撰写本文时，Google Gemini提供此功能，缓存的输入标记比普通输入标记有75%的折扣，但你需要额外支付缓存存储费用（截至撰写时，每小时每百万标记1.00美元）。Anthropic提供承诺高达90%成本节省（缓存上下文越长，节省越高）和高达75%延迟减少的提示缓存。表9-3显示了提示缓存对不同场景成本和延迟的影响。

表9-3. 提示缓存减少的成本和延迟。信息来自Anthropic(2024)。

用例	不缓存的延迟 (首个标记时间)	缓存的延迟 (首个标记时间)	成本减少

与书籍聊天(100,000标记缓存提示)	11.5秒	2.4秒(-79%)	-90%
多样本提示(10,000标记提示)	1.6秒	1.1秒(-31%)	-86%
多轮对话(具有长系统提示的10轮对话)	~10秒	~2.5秒(-75%)	-53%

并行性

加速器是为并行处理设计的，并行策略是高性能计算的骨干。许多新的并行化策略正在开发中。本节仅涵盖其中一些作为参考。可以应用于所有模型的两个并行化策略系列是数据并行和模型并行。专门应用于LLM的策略系列是上下文和序列并行。优化技术可能涉及多种并行策略。

副本并行是最直接的实现策略。它只是创建你想要服务的模型的多个副本。更多副本允许你同时处理更多请求，可能以使用更多芯片为代价。尝试将不同大小的模型放在不同芯片上是一个装箱问题，随着更多模型、更多副本和更多芯片，这可能变得复杂。

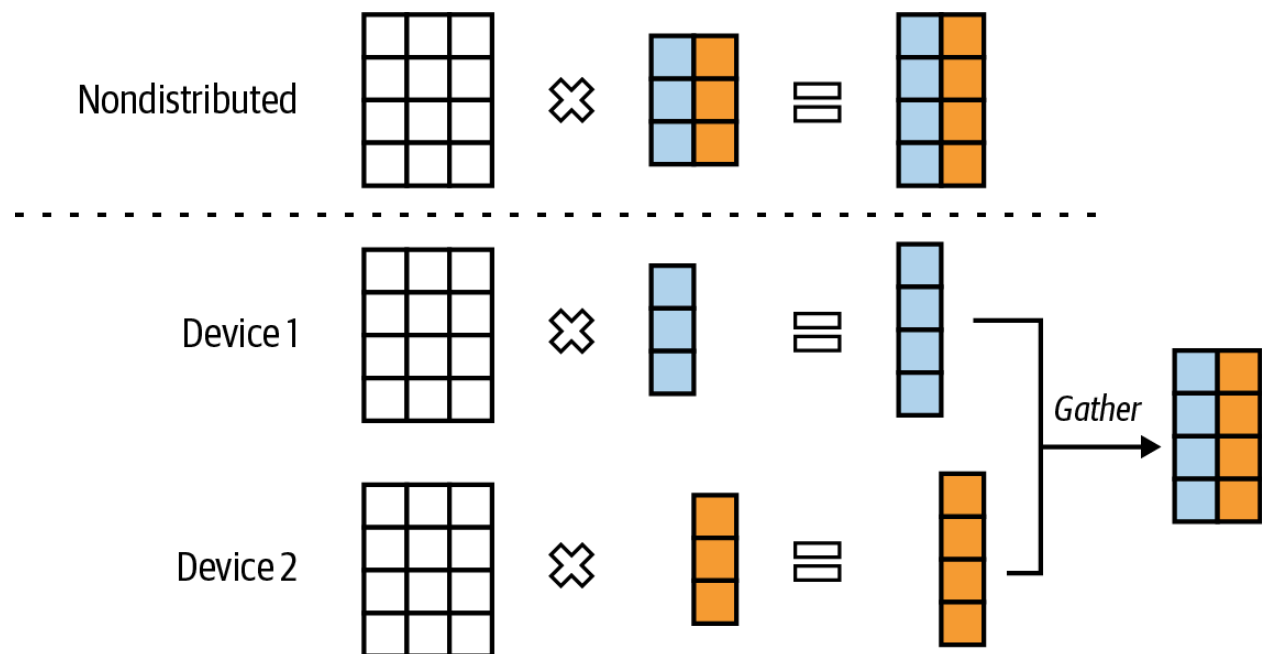
假设你有不同大小的模型(例如，8B、13B、34B和70B参数)和不同内存能力的GPU(例如，24 GB、40 GB、48 GB和80 GB)。为简单起见，假设所有模型都是相同精度，8位：

- 如果你有固定数量的芯片，你需要决定为每个模型创建多少副本以及为每个副本使用什么GPU，以最大化你的指标。例如，你应该在40 GB GPU上放置三个13B模型，还是应该为一个34B模型保留这个GPU？
- 如果你有固定数量的模型副本，你需要决定获取什么芯片以最小化成本。然而，这种情况很少发生。

通常，你的模型太大，无法放入一台机器。模型并行指的是将同一模型分布在多台机器上的做法。使用模型并行时，将模型放入芯片可能变得更加复杂。

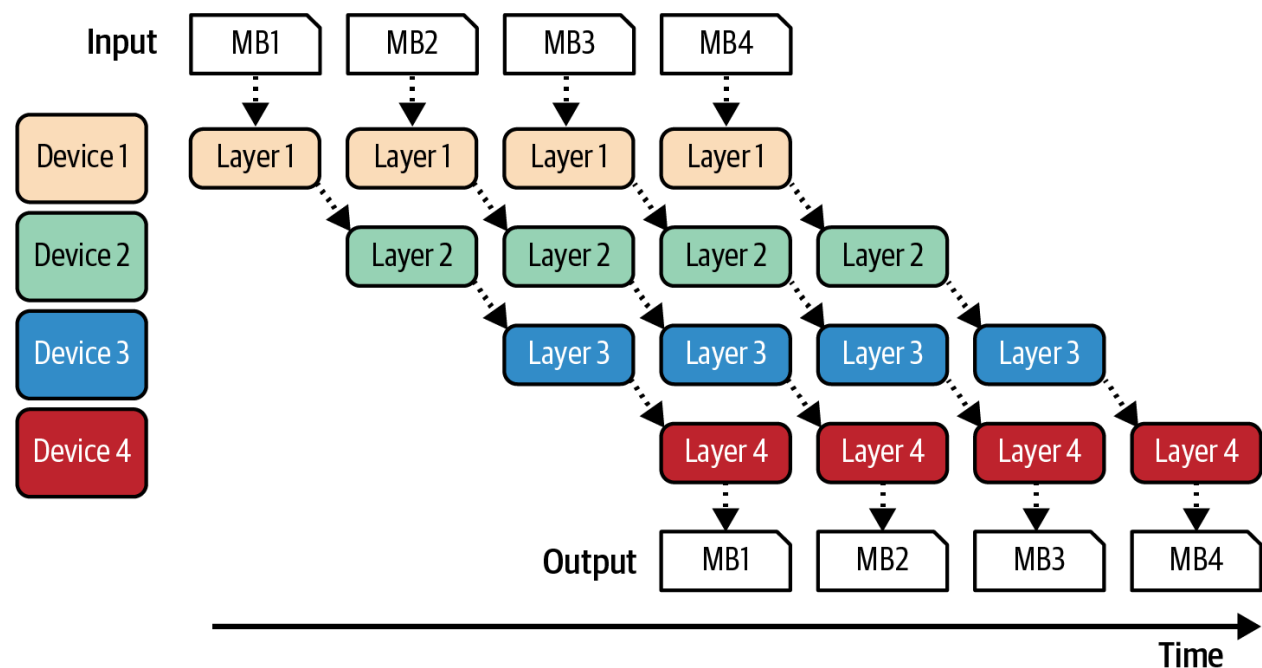
有几种方法可以分割模型。推理最常见的方法是张量并行，也称为算子内并行。推理涉及多维张量上的一系列操作，如矩阵乘法。在这种方法中，参与操作的张量在多个设备上分区，有效地将这个操作分成更小的部分并行执行，从而加速计算。例如，当乘两个矩阵时，可以按列分割其中一个矩阵，如图9-18所示。

张量并行提供两个好处。首先，它使得服务不适合单台机器的大模型成为可能。其次，它减少了延迟。然而，由于额外的通信开销，延迟好处可能会减少。



【图9-18. 矩阵乘法的张量并行。】

分割模型的另一种方式是流水线并行，它涉及将模型的计算分成不同阶段，并将每个阶段分配给不同的设备。当数据流经模型时，每个阶段处理一部分，而其他阶段处理后续部分，使计算重叠。图9-19显示了四台机器上的流水线并行是什么样子。



【图9-19. 流水线并行使模型分割能并行执行。】

图9-19显示批次可以分成更小的微批次。在一台机器上处理微批次后，其输出传递到下一台机器上的模型下一部分。

虽然流水线并行使在多台机器上服务大模型成为可能，但由于流水线阶段之间的额外通信，它增加了每个请求的总延迟。因此，对于具有严格延迟要求的应用程序，通常避免流水线并行，而倾向于副本并行。然而，流水线并行常用于训练，因为它可以帮助提高吞吐量。

两种不太常见但可能值得简要提及以说明技术多样性的技术是上下文并行和序列并行。它们都是为使长输入序列处理更高效而开发的，包括上下文并行和序列并行。

在上下文并行中，输入序列本身在不同设备上分割以单独处理。例如，输入的前半部分在机器1上处理，后半部分在机器2上处理。

在序列并行中，整个输入所需的操作在机器之间分割。例如，如果输入需要注意力和前馈计算，注意力可能在机器1上处理，而前馈在机器2上处理。

总结

模型的可用性在很大程度上取决于其推理成本和延迟。更便宜的推理使AI驱动的决策更加可负担，而更快的推理使将AI集成到更多应用程序中成为可能。鉴于推理优化的巨大潜在影响，它吸引了许多有才华的人，他们不断提出创新方法。

在开始提高效率之前，我们需要了解如何衡量效率。本章首先介绍了延迟、吞吐量和利用率的常见效率指标。对于基于语言模型的推理，延迟可以分解为首个标记的时间(TTFT)，受预填充阶段影响，以及每输出标记的时间(TPOT)，受解码阶段影响。吞吐量指标与成本直接相关。延迟和吞吐量之间存在权衡。如果你接受增加的延迟，可能可以降低成本，而减少延迟通常涉及增加成本。

模型能够多高效地运行取决于它运行的硬件。因此，本章还提供了AI硬件的快速概述，以及在不同加速器上优化模型所需的内容。

然后，本章继续介绍了不同的推理优化技术。鉴于模型API的可用性，大多数应用程序开发人员将使用这些API及其内置优化，而不是自己实现这些技术。虽然这些技术可能不会与所有应用程序开发人员相关，但我相信了解可能的技术对于评估模型API的效率会有所帮助。

本章还重点关注了模型层面和推理服务层面的优化。模型层面的优化通常需要改变模型本身，这可能导致模型行为的变化。而推理服务层面的优化通常保持模型不变，只改变其服务方式。

模型层面的技术包括模型无关技术，如量化和蒸馏。不同的模型架构需要自己的优化。例如，因为transformer模型的关键瓶颈在于注意力机制，许多优化技术涉及使注意力更高效，包括KV缓存管理和编写注意力内核。自回归语言模型的一个重大瓶颈在于其自回归解码过程，因此也开发了许多技术来解决它。

推理服务层面的技术包括各种批处理和并行策略。还有专门为自回归语言模型开发的技术，包括预填充/解码解耦和提示缓存。

优化技术的选择取决于你的工作负载。例如，KV缓存对于长上下文的工作负载明显比短上下文的更重要。而提示缓存对于涉及长而重叠的提示段或多轮对话的工作负载至关重要。选择也取决于你的性能要求。例如，如果低延迟比成本更高优先级，你可能希望扩大副本并行。虽然更多副本需要额外的机器，但每台机器处理的请求更少，使其能够为每个请求分配更多资源，从而提高响应时间。

然而，在各种用例中，最有影响力的技术通常是量化（通常在模型间表现良好）、张量并行（既减少延迟又使服务更大模型成为可能）、副本并行（相对容易实现）和注意力机制优化（可以显著加速transformer模型）。

推理优化结束了本书中涵盖的模型适应技术列表。下一章将探讨如何将这些技术整合到一个连贯的系统中。